

(54) **METHOD AND APPARATUS FOR CODING IMAGE ON BASIS OF TRANSFORM**

(71) Applicant: **LG ELECTRONICS INC.**, Seoul (KR)

(72) Inventors: **Moonmo KOO**, Seoul (KR);
Seunghwan KIM, Seoul (KR);
Jaehyun LIM, Seoul (KR)

(73) Assignee: **LG ELECTRONICS INC.**, Seoul (KR)

(21) Appl. No.: **19/214,898**

(22) Filed: **May 21, 2025**

Related U.S. Application Data

- (63) Continuation of application No. 18/411,992, filed on Jan. 12, 2024, now Pat. No. 12,341,993, which is a continuation of application No. 18/100,277, filed on Jan. 23, 2023, now Pat. No. 11,997,315, which is a continuation of application No. 17/572,282, filed on Jan. 10, 2022, now Pat. No. 11,611,776, which is a continuation of application No. PCT/KR2020/009124, filed on Jul. 10, 2020.
- (60) Provisional application No. 62/873,818, filed on Jul. 12, 2019, provisional application No. 62/873,896, filed on Jul. 13, 2019.

Publication Classification

- (51) **Int. Cl.**
H04N 19/60 (2014.01)
H04N 19/122 (2014.01)
H04N 19/124 (2014.01)
H04N 19/132 (2014.01)
H04N 19/176 (2014.01)
H04N 19/18 (2014.01)
H04N 19/70 (2014.01)
- (52) **U.S. Cl.**
CPC **H04N 19/60** (2014.11); **H04N 19/122** (2014.11); **H04N 19/124** (2014.11); **H04N 19/132** (2014.11); **H04N 19/176** (2014.11); **H04N 19/18** (2014.11); **H04N 19/70** (2014.11)

(57) **ABSTRACT**

An image decoding method according to the present document may comprise the steps of: deriving a first variable indicating whether there is a valid coefficient in a region excluding a DC region from a current block; deriving a second variable indicating whether there is a valid coefficient in a second region excluding a first region formed at the upper left end of the current block; when the first variable indicates that the valid coefficient exists in the region excluding the DC region, and the second variable indicates that the valid coefficient does not exist in the second region, parsing an LFNST index from the bitstream; and applying an LFNST matrix derived on the basis of the LFNST index to transform coefficients in the first region, to derive the modified transform coefficients.

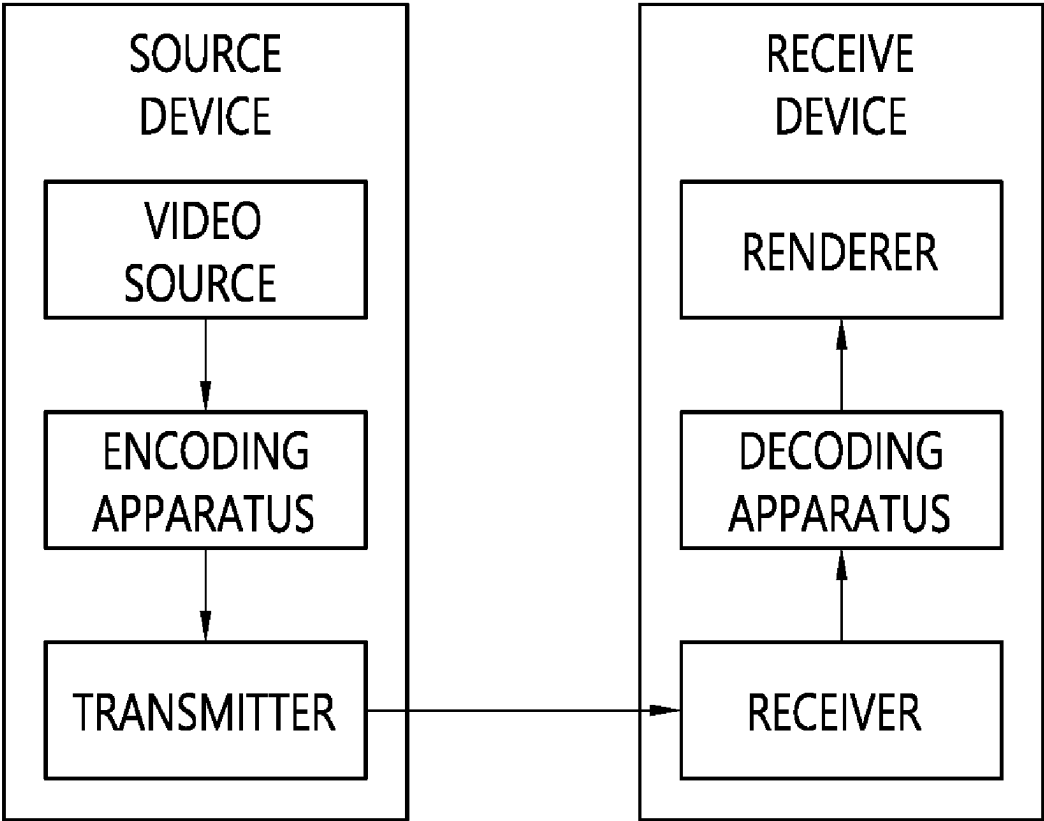


FIG. 1

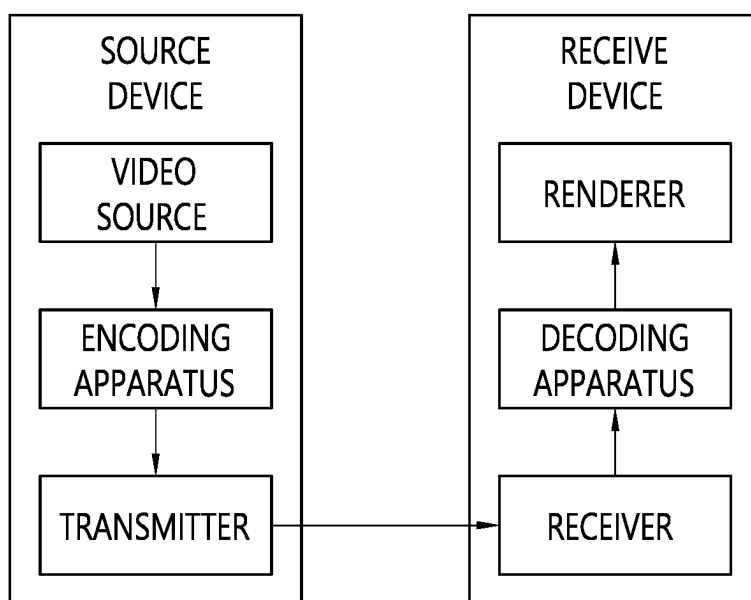


FIG. 2

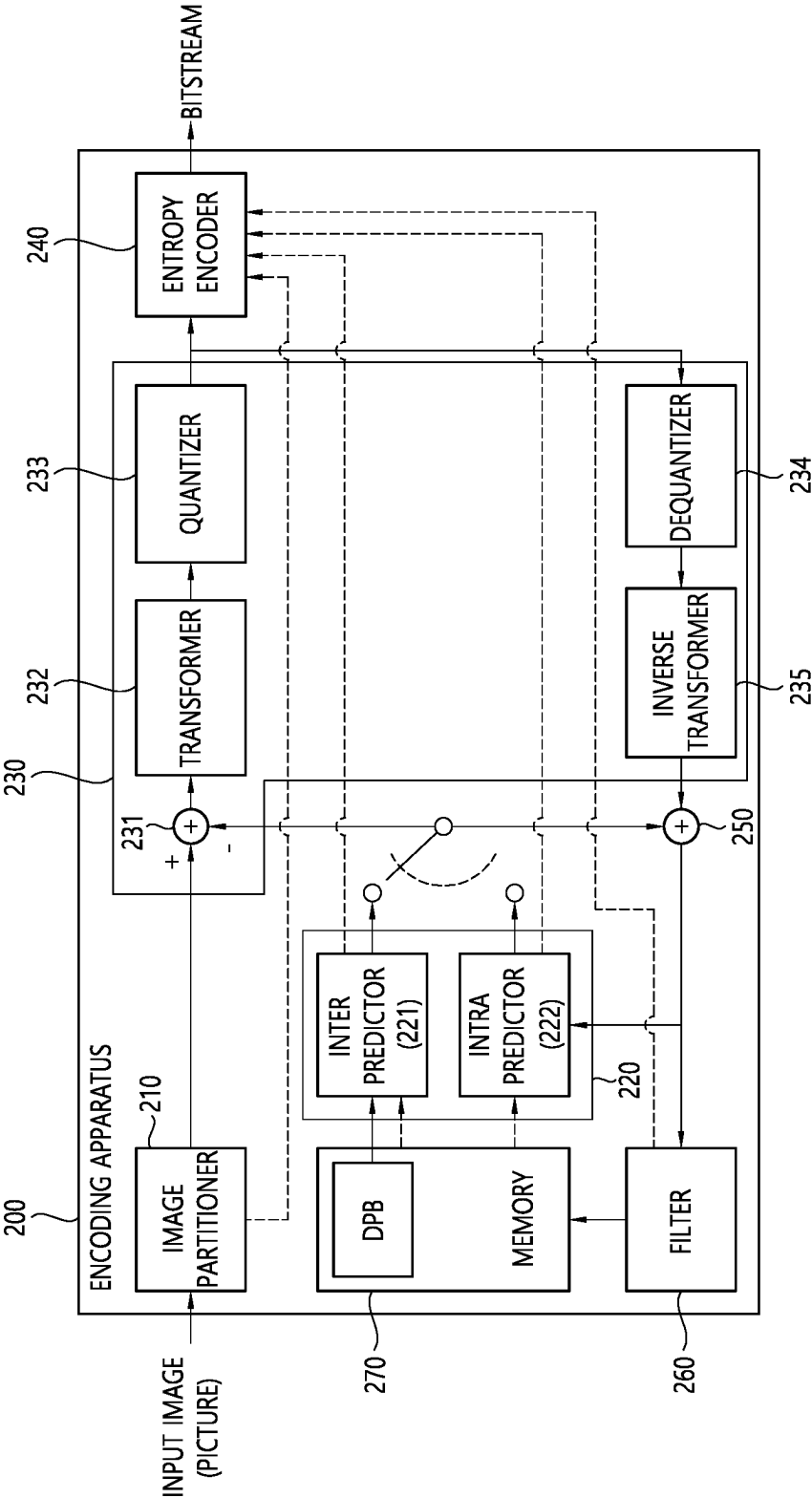


FIG. 3

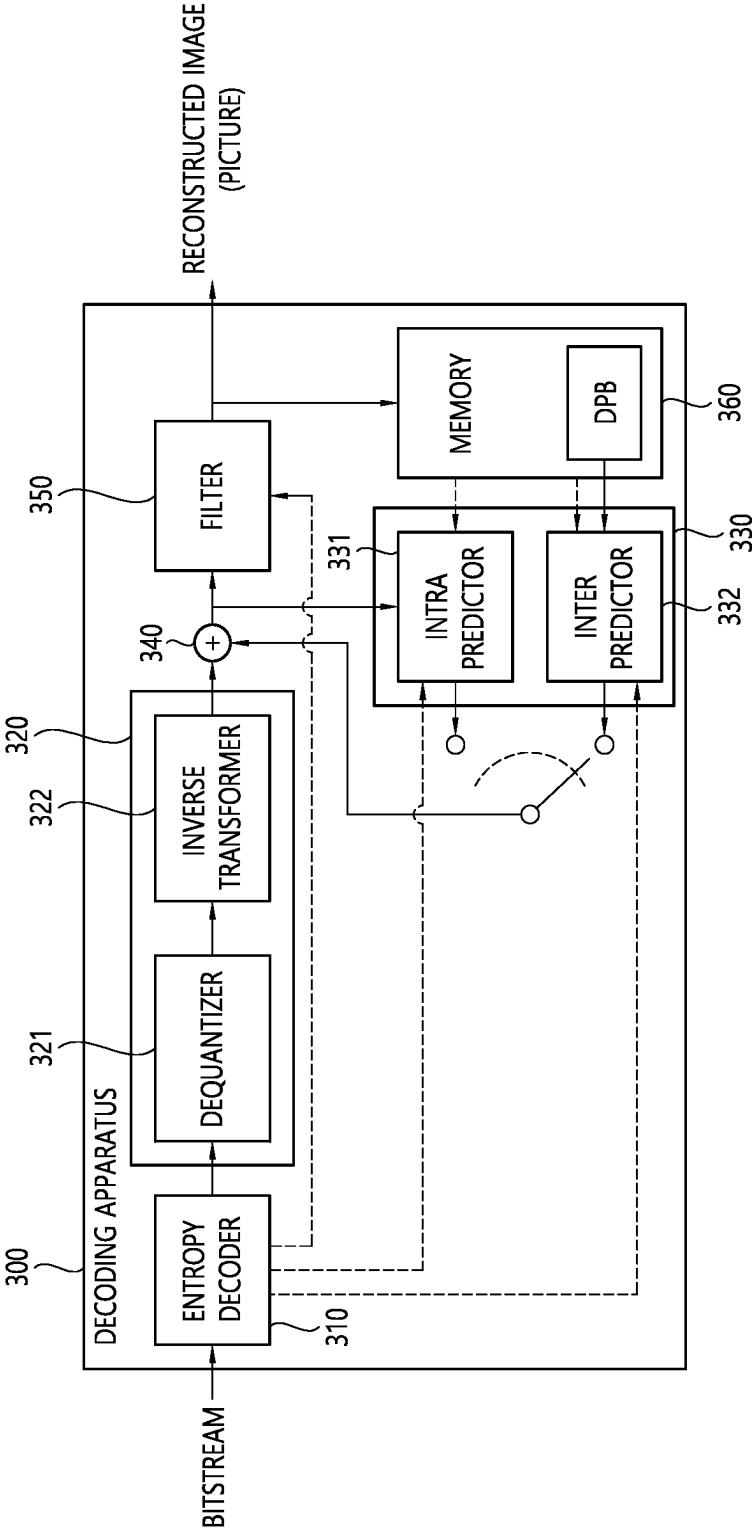


FIG. 4

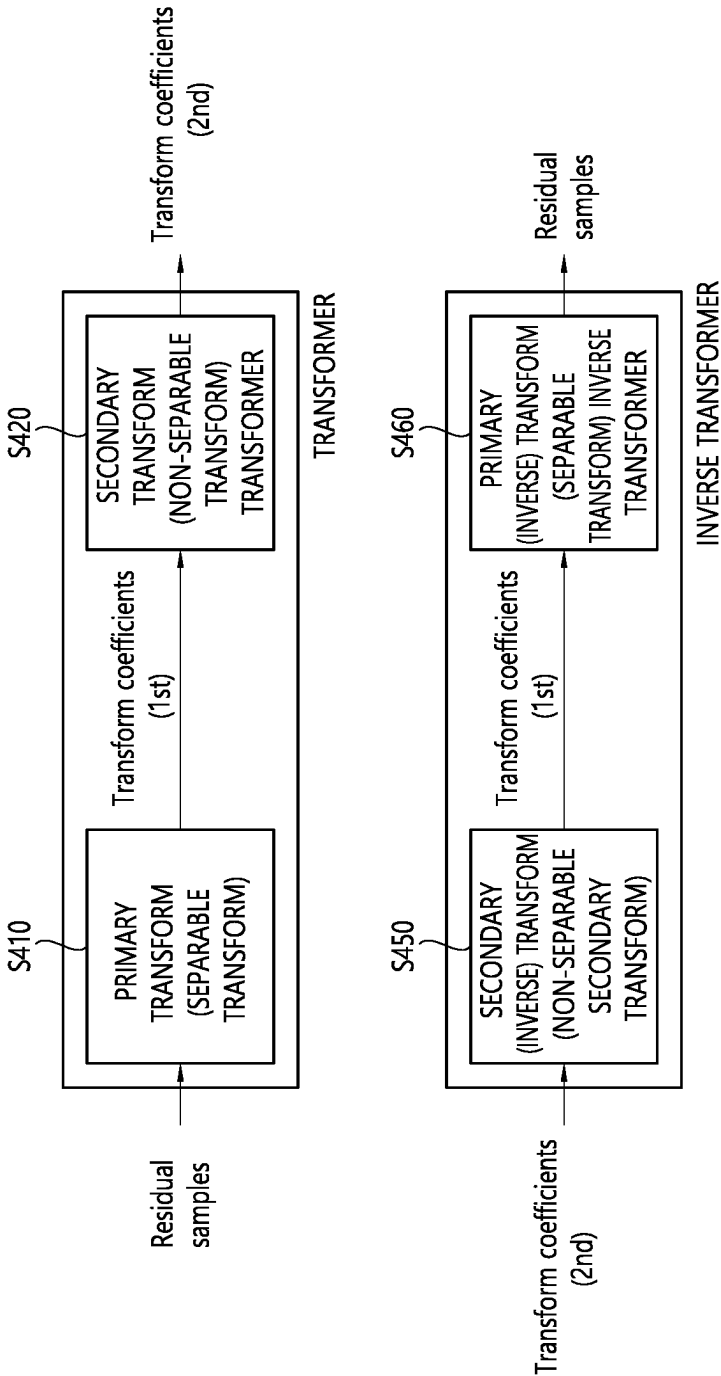


FIG. 5

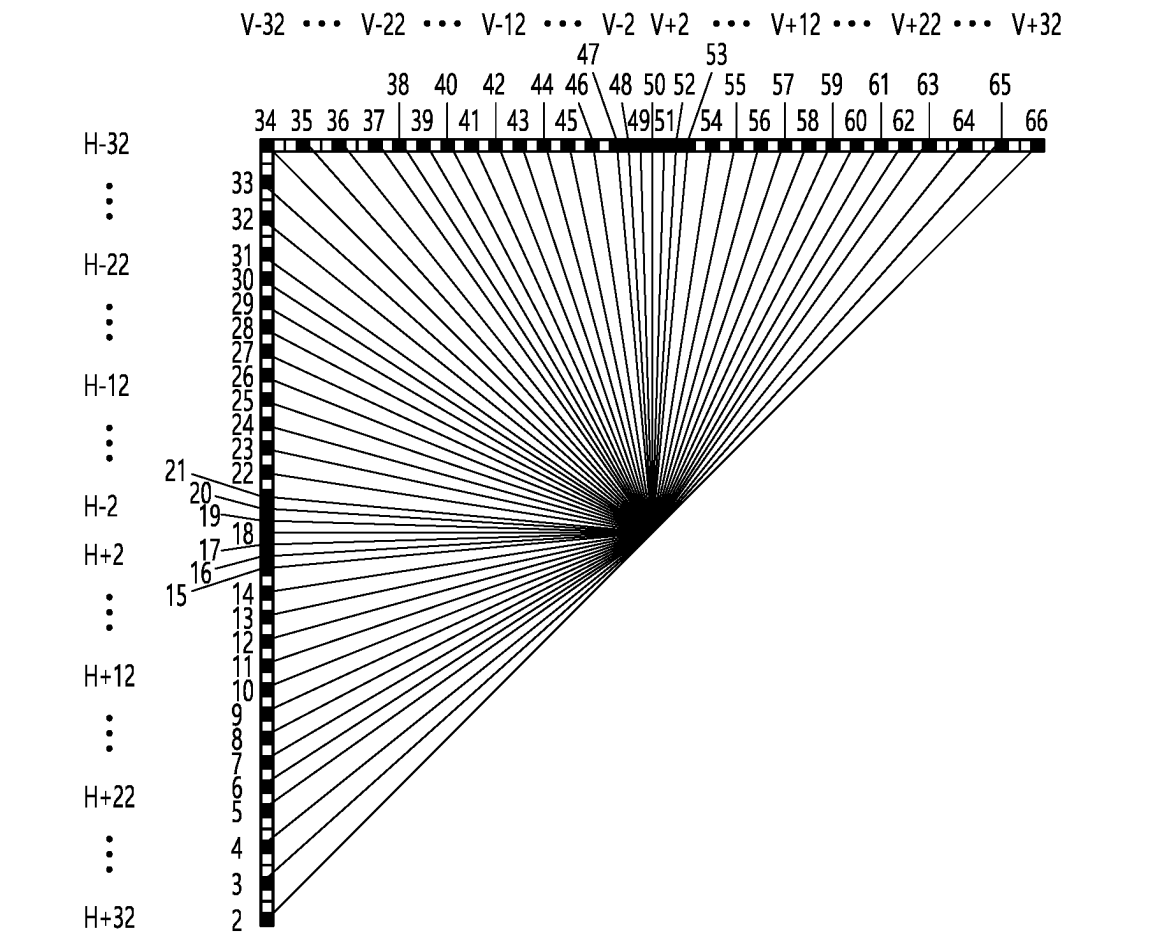


FIG. 6

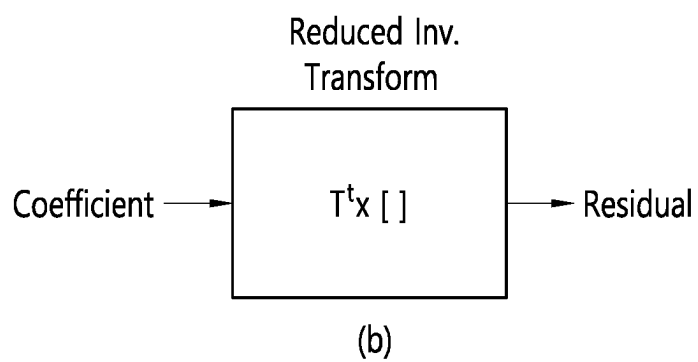
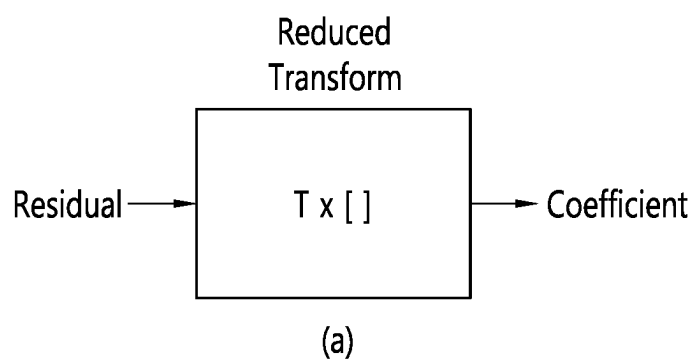


FIG. 7

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32
33	34	35	36				
37	38	39	40				
41	42	43	44				
45	46	47	48				

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

(a)

1	9	17	25	33	37	41	45
2	10	18	26	34	38	42	46
3	11	19	27	35	39	43	47
4	12	20	28	36	40	44	48
5	13	21	29				
6	14	22	30				
7	15	23	31				
8	16	24	32				

1	5	9	13
2	6	10	14
3	7	11	15
4	8	12	16

(b)

FIG. 8

1	3	6	10
2	5	9	13
4	8	12	15
7	11	14	16

(a)

1	3	6	
2	5		
4	8		
7			

(b)

FIG. 9

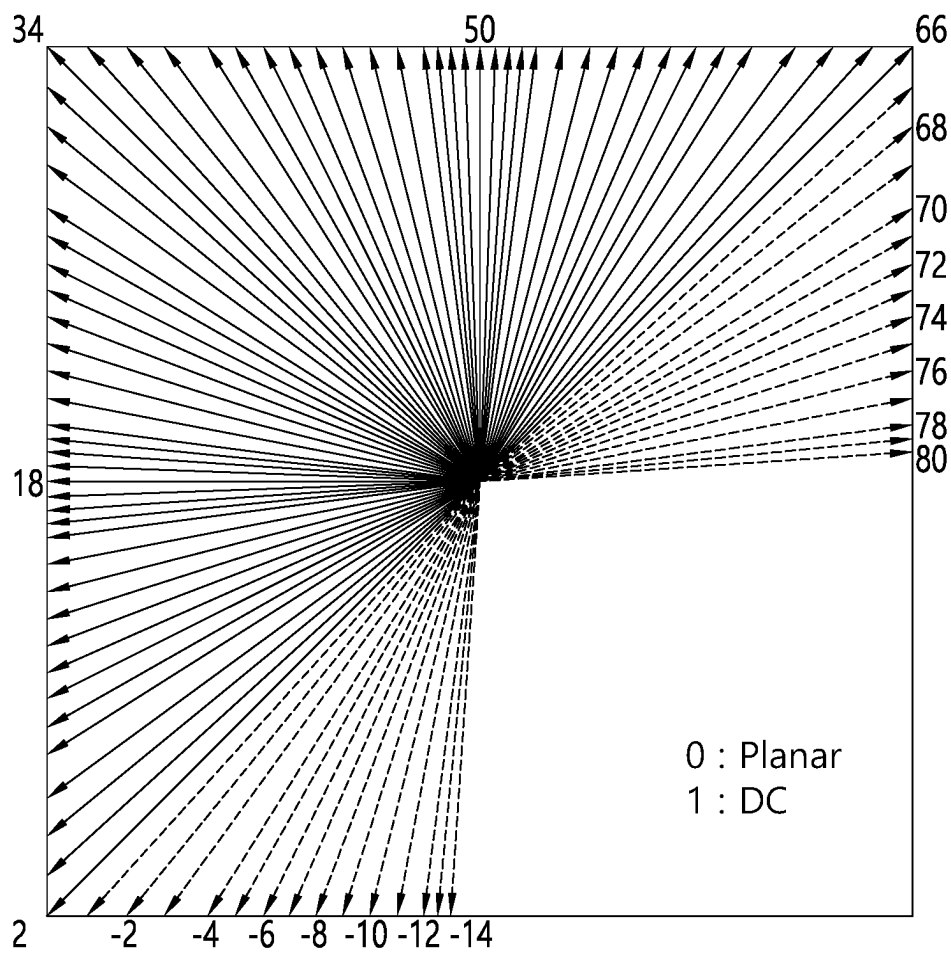
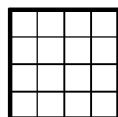
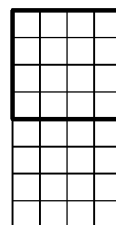
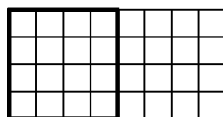


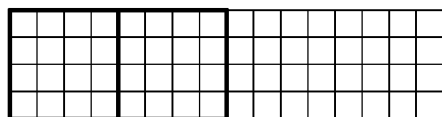
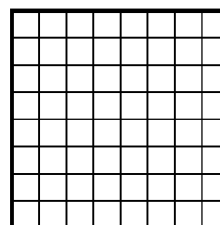
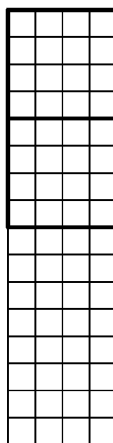
FIG. 10



(a) 4x4



(b) 8x4 / 4x8

(c) 4xN / Nx4, when $N \geq 16$ 

(d) 8x8

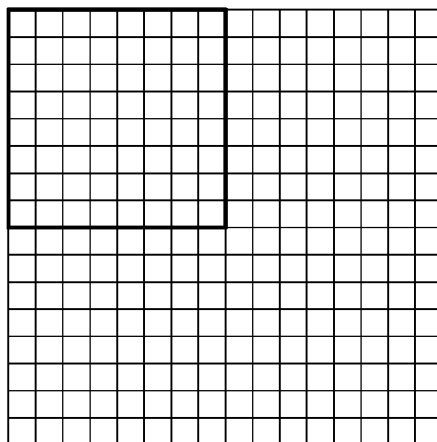
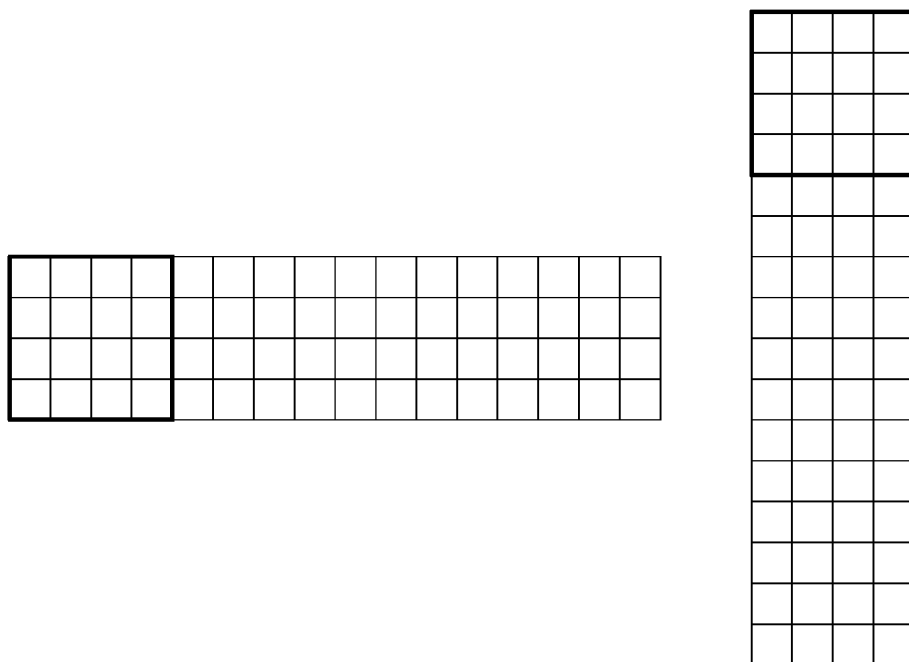
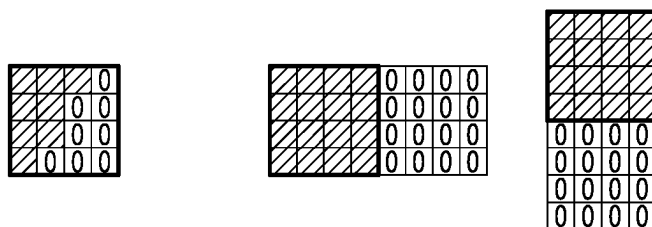
(e) $M \times N (M \geq 8, N \geq 8, M > 8 \text{ or } N > 8)$

FIG. 12



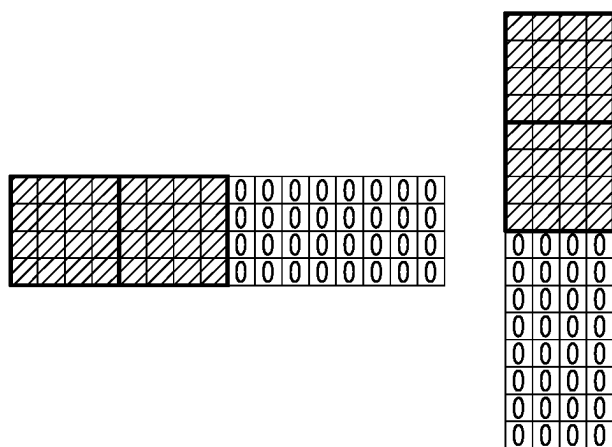
$4 \times N / N \times 4$, when $N \geq 16$

FIG. 13

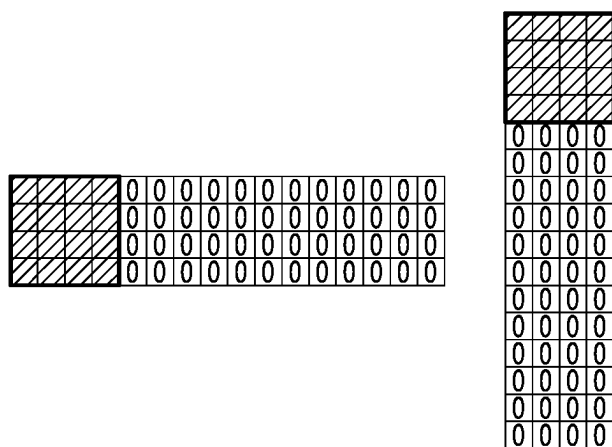


(a) 4x4

(b) 8x4 / 4x8



(c) 4xN / Nx4, when $N \geq 16$



(d) 4xN / Nx4, when $N \geq 16$

FIG. 16

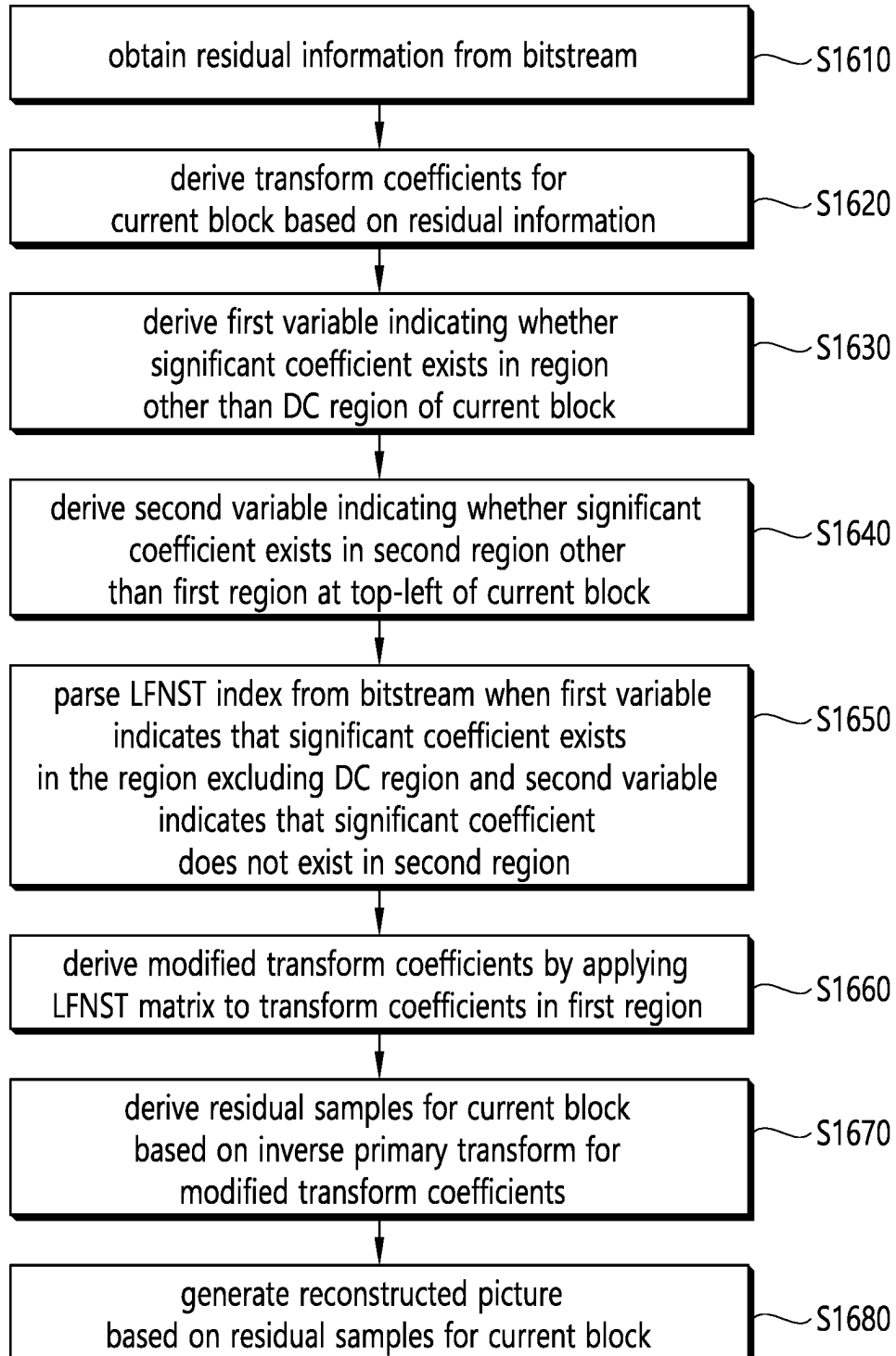


FIG. 17

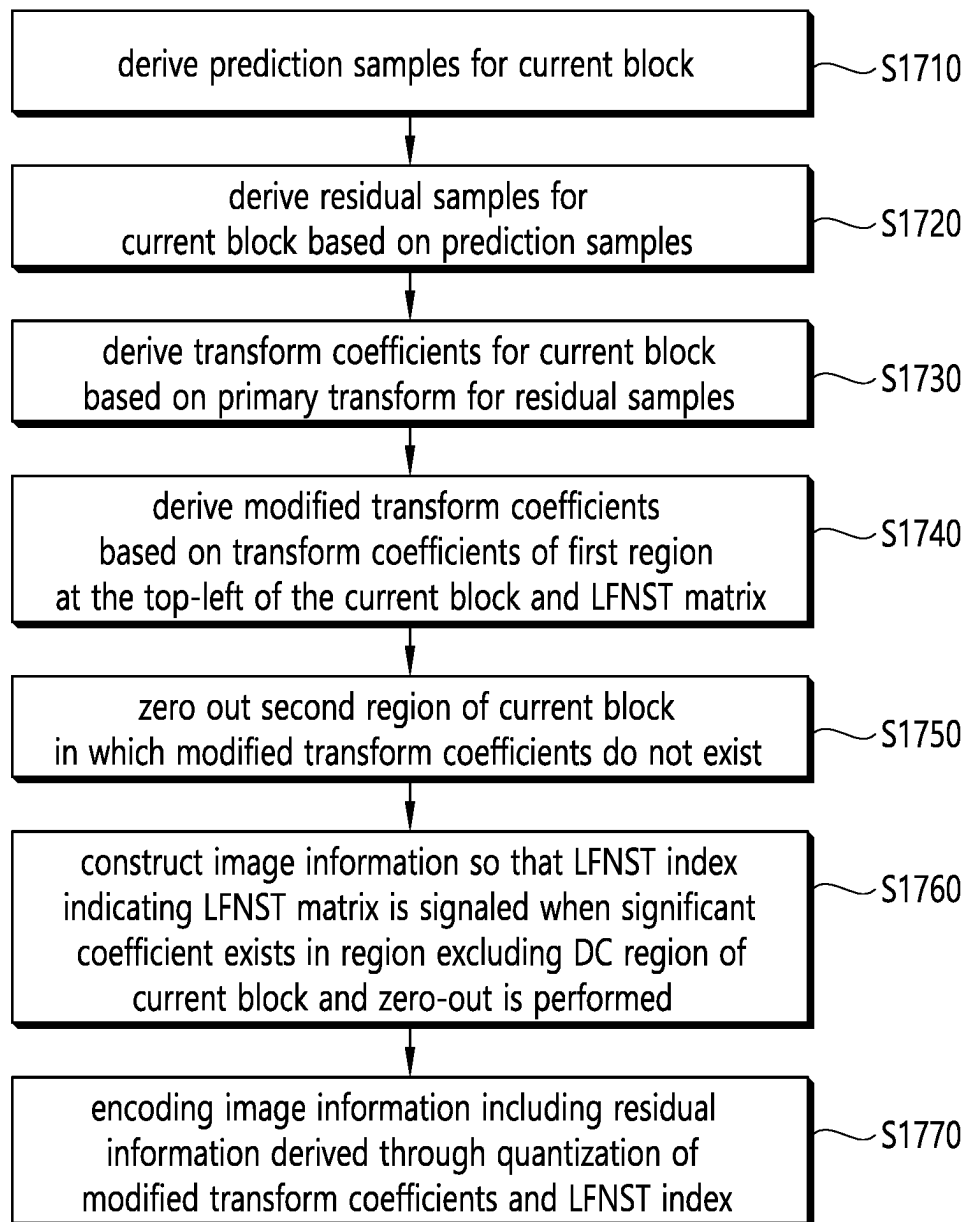
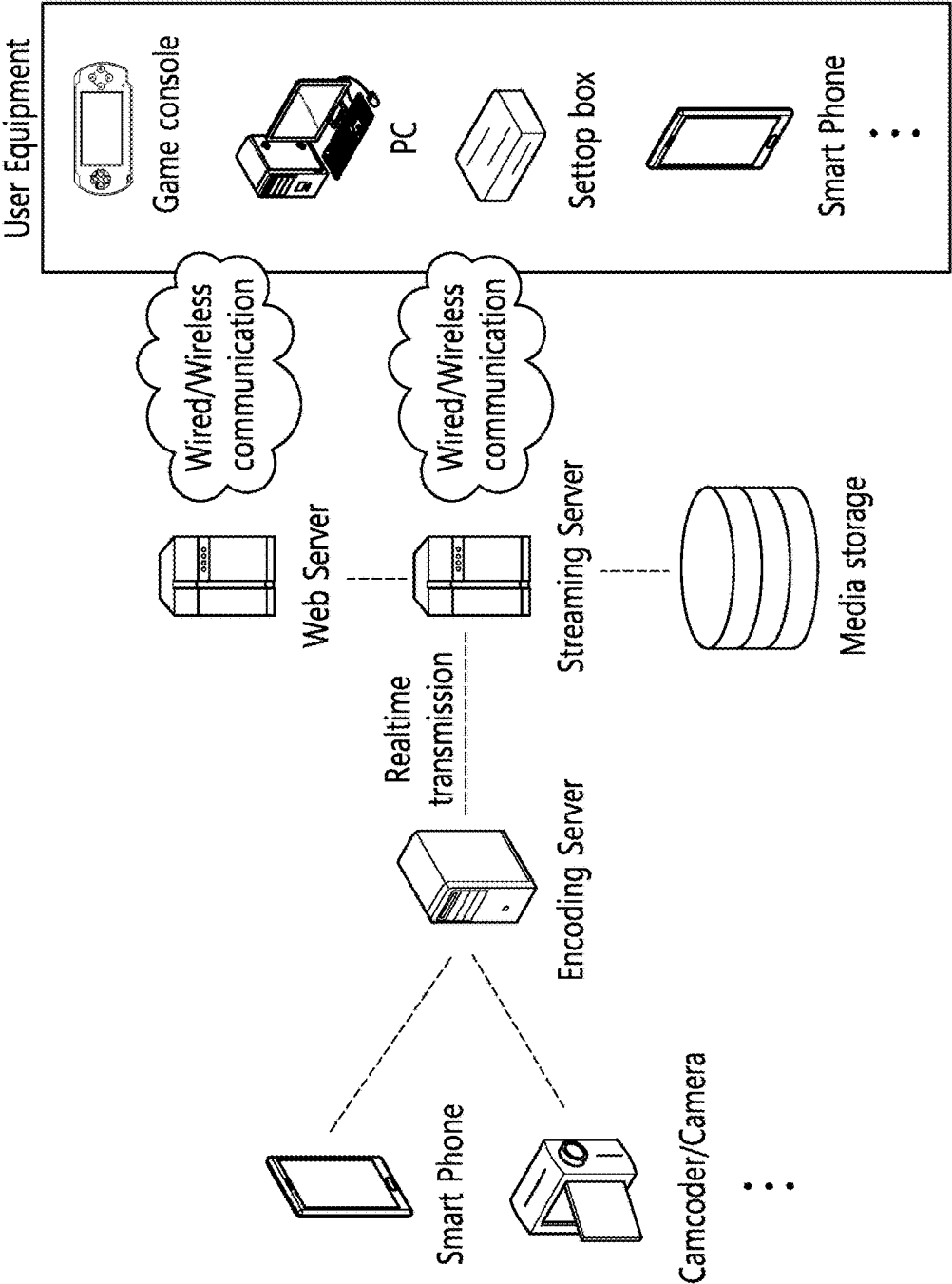


FIG. 18



METHOD AND APPARATUS FOR CODING IMAGE ON BASIS OF TRANSFORM

CROSS-REFERENCE TO RELATED APPLICATIONS

[0001] This application is a Continuation of U.S. application Ser. No. 18/411,992 filed Jan. 12, 2024, now pending, which is a Continuation of U.S. application Ser. No. 18/100,277, filed Jan. 23, 2023, (now U.S. Pat. No. 11,997,315 issued May 28, 2024), which is a Continuation of U.S. application Ser. No. 17/572,282, filed Jan. 10, 2022 (now U.S. Pat. No. 11,611,776 issued on Mar. 21, 2023), which is a Continuation of International Application No. PCT/KR2020/009124, filed on Jul. 10, 2020, which claims the benefit of U.S. Provisional Application No. 62/873,818, filed on Jul. 12, 2019 and, 62/873,896, filed Jul. 13, 2019 the contents of which are all hereby incorporated by reference herein in their entirety.

TECHNICAL FIELD

[0002] The present disclosure relates to an image coding technique and, more particularly, to a method and an apparatus for coding an image based on transform in an image coding system.

RELATED ART

[0003] Nowadays, the demand for high-resolution and high-quality images/videos such as 4K, 8K or more ultra high definition (UHD) images/videos has been increasing in various fields. As the image/video data becomes higher resolution and higher quality, the transmitted information amount or bit amount increases as compared to the conventional image data. Therefore, when image data is transmitted using a medium such as a conventional wired/wireless broadband line or image/video data is stored using an existing storage medium, the transmission cost and the storage cost thereof are increased.

[0004] Further, nowadays, the interest and demand for immersive media such as virtual reality (VR), artificial reality (AR) content or hologram, or the like is increasing, and broadcasting for images/videos having image features different from those of real images, such as a game image is increasing.

[0005] Accordingly, there is a need for a highly efficient image/video compression technique for effectively compressing and transmitting or storing, and reproducing information of high resolution and high quality images/videos having various features as described above.

SUMMARY

[0006] A technical aspect of the present disclosure is to provide a method and an apparatus for increasing image coding efficiency.

[0007] Another technical aspect of the present disclosure is to provide a method and an apparatus for increasing efficiency in transform index coding.

[0008] Still another technical aspect of the present disclosure is to provide an image coding method and apparatus using LFNST.

[0009] Still another technical aspect of the present disclosure is to provide an image coding method and apparatus for zero-out performed when LFNST is applied.

[0010] According to an embodiment of the present disclosure, there is provided an image decoding method performed by a decoding apparatus. The method may include: obtaining residual information from a bitstream; deriving transform coefficients for a current block based on the residual information; deriving a first variable indicating whether a significant coefficient exists in a region excluding a DC region of the current block; deriving a second variable indicating whether a significant coefficient exists in a second region other than a first region at the top-left of the current block; parsing an LFNST index from the bitstream when the first variable indicates that the significant coefficient exists in the region excluding the DC region and the second variable indicates that the significant coefficient does not exist in the second region; deriving residual samples for the current block based on an inverse primary transform for the modified transform coefficients; and generating a reconstructed picture based on the residual samples for the current block.

[0011] The first variable is derived as 0 when the index of a subblock including a last significant coefficient in the current block is 0 and the position of the last significant coefficient in the subblock is greater than 0, and the LFNST index is parsed when the first variable is 0.

[0012] The LFNST index is parsed when the first variable is 0 the first variable is changed to 0 when the significant coefficient exists in the region excluding the DC region.

[0013] The second variable is derived as 0 when the index of the sub-block including the last significant coefficient in the current block is greater than 0 and the width and height of the current block are 4 or more, and the LFNST index is not parsed when the second variable is 0.

[0014] The second variable is derived as 0 when the size of the current block is 4×4 or 8×8 and the position of the last significant coefficient is greater than 7, and the LFNST index is not parsed when the second variable is 0.

[0015] The second variable is initially set to 1, and the second variable is changed to 0 when the significant coefficient exists in the second region.

[0016] According to another embodiment of the present disclosure, there is provided an image encoding method performed by an encoding apparatus. The method may include: deriving prediction samples for a current block; deriving residual samples for the current block based on the prediction sample; deriving transform coefficients for the current block based on a primary transform for the residual samples; deriving modified transform coefficients for the current block based on the transform coefficients of a first region at the top-left of the current block and a predetermined LFNST matrix; zeroing out a second region of the current block in which the modified transform coefficients do not exist; constructing image information so that an LFNST index indicating the LFNST matrix is signaled when a significant coefficient exists in a region excluding a DC region of the current block and the zeroing-out is performed, and encoding the image information including residual information derived through quantization of the modified transform coefficients and the LFNST index.

[0017] According to still another embodiment of the present disclosure, there may be provided a digital storage medium that stores image data including encoded image information and a bitstream generated according to an image encoding method performed by an encoding apparatus.

[0018] According to yet another embodiment of the present disclosure, there may be provided a digital storage medium that stores image data including encoded image information and a bitstream to cause a decoding apparatus to perform the image decoding method.

[0019] According to the present disclosure, it is possible to increase overall image/video compression efficiency.

[0020] According to the present disclosure, it is possible to increase efficiency in transform index coding.

[0021] A technical aspect of the present disclosure may provide an image coding method and apparatus using LFNST.

[0022] A technical aspect of the present disclosure may provide an image coding method and apparatus for zero-out performed when LFNST is applied.

[0023] The effects that can be obtained through specific examples of the present disclosure are not limited to the effects listed above. For example, there may be various technical effects that a person having ordinary skill in the related art can understand or derive from the present disclosure. Accordingly, specific effects of the present disclosure are not limited to those explicitly described in the present disclosure and may include various effects that can be understood or derived from the technical features of the present disclosure.

BRIEF DESCRIPTION OF THE DRAWINGS

[0024] FIG. 1 schematically illustrates an example of a video/image coding system to which the present disclosure is applicable.

[0025] FIG. 2 is a diagram schematically illustrating a configuration of a video/image encoding apparatus to which the present disclosure is applicable.

[0026] FIG. 3 is a diagram schematically illustrating a configuration of a video/image decoding apparatus to which the present disclosure is applicable.

[0027] FIG. 4 is schematically illustrates a multiple transform scheme according to an embodiment of the present document.

[0028] FIG. 5 exemplarily shows intra directional modes of 65 prediction directions.

[0029] FIG. 6 is a diagram for explaining RST according to an embodiment of the present.

[0030] FIG. 7 is a diagram illustrating a sequence of arranging output data of a forward primary transformation into a one-dimensional vector according to an example.

[0031] FIG. 8 is a diagram illustrating a sequence of arranging output data of a forward secondary transform into a two-dimensional vector according to an example.

[0032] FIG. 9 is a diagram illustrating wide-angle intra prediction modes according to an embodiment of the present document.

[0033] FIG. 10 is a diagram illustrating a block shape to which LFNST is applied.

[0034] FIG. 11 is a diagram illustrating an arrangement of output data of a forward LFNST according to an example.

[0035] FIG. 12 is a diagram illustrating that the number of output data for a forward LFNST is limited to a maximum of 16 according to an example.

[0036] FIG. 13 is a diagram illustrating zero-out in a block to which 4x4 LFNST is applied according to an example.

[0037] FIG. 14 is a diagram illustrating zero-out in a block to which 8x8 LFNST is applied according to an example.

[0038] FIG. 15 is a diagram illustrating zero-out in a block to which 8x8 LFNST is applied according to another example.

[0039] FIG. 16 is a flowchart for explaining an image decoding method according to an example.

[0040] FIG. 17 is a flowchart for explaining an image encoding method according to an example.

[0041] FIG. 18 illustrates the structure of a content streaming system to which the present disclosure is applied.

DESCRIPTION OF EXEMPLARY EMBODIMENTS

[0042] While the present disclosure may be susceptible to various modifications and include various embodiments, specific embodiments thereof have been shown in the drawings by way of example and will now be described in detail. However, this is not intended to limit the present disclosure to the specific embodiments disclosed herein. The terminology used herein is for the purpose of describing specific embodiments only, and is not intended to limit technical idea of the present disclosure. The singular forms may include the plural forms unless the context clearly indicates otherwise. The terms such as “include” and “have” are intended to indicate that features, numbers, steps, operations, elements, components, or combinations thereof used in the following description exist, and thus should not be understood as that the possibility of existence or addition of one or more different features, numbers, steps, operations, elements, components, or combinations thereof is excluded in advance.

[0043] Meanwhile, each component on the drawings described herein is illustrated independently for convenience of description as to characteristic functions different from each other, and however, it is not meant that each component is realized by a separate hardware or software. For example, any two or more of these components may be combined to form a single component, and any single component may be divided into plural components. The embodiments in which components are combined and/or divided will belong to the scope of the patent right of the present disclosure as long as they do not depart from the essence of the present disclosure.

[0044] Hereinafter, preferred embodiments of the present disclosure will be explained in more detail while referring to the attached drawings. In addition, the same reference signs are used for the same components on the drawings, and repeated descriptions for the same components will be omitted.

[0045] This document relates to video/image coding. For example, the method/example disclosed in this document may relate to a VVC (Versatile Video Coding) standard (ITU-T Rec. H.266), a next-generation video/image coding standard after VVC, or other video coding related standards (e.g., HEVC (High Efficiency Video Coding) standard (ITU-T Rec. H.265), EVC (essential video coding) standard, AVS2 standard, etc.).

[0046] In this document, a variety of embodiments relating to video/image coding may be provided, and, unless specified to the contrary, the embodiments may be combined to each other and be performed.

[0047] In this document, a video may mean a set of a series of images over time. Generally a picture means a unit representing an image at a specific time zone, and a slice/tile is a unit constituting a part of the picture. The slice/tile may

include one or more coding tree units (CTUs). One picture may be constituted by one or more slices/tiles. One picture may be constituted by one or more tile groups. One tile group may include one or more tiles.

[0048] A pixel or a pel may mean a smallest unit constituting one picture (or image). Also, 'sample' may be used as a term corresponding to a pixel. A sample may generally represent a pixel or a value of a pixel, and may represent only a pixel/pixel value of a luma component or only a pixel/pixel value of a chroma component. Alternatively, the sample may refer to a pixel value in the spatial domain, or when this pixel value is converted to the frequency domain, it may refer to a transform coefficient in the frequency domain.

[0049] A unit may represent the basic unit of image processing. The unit may include at least one of a specific region and information related to the region. One unit may include one luma block and two chroma (e.g., cb, cr) blocks. The unit and a term such as a block, an area, or the like may be used in place of each other according to circumstances. In a general case, an MxN block may include a set (or an array) of samples (or sample arrays) or transform coefficients consisting of M columns and N rows.

[0050] In this document, the term "/" and "," should be interpreted to indicate "and/or." For instance, the expression "A/B" may mean "A and/or B." Further, "A, B" may mean "A and/or B." Further, "A/B/C" may mean "at least one of A, B, and/or C." Also, "A/B/C" may mean "at least one of A, B, and/or C."

[0051] Further, in the document, the term "or" should be interpreted to indicate "and/or." For instance, the expression "A or B" may include 1) only A, 2) only B, and/or 3) both A and B. In other words, the term "or" in this document should be interpreted to indicate "additionally or alternatively."

[0052] In the present disclosure, "at least one of A and B" may mean "only A", "only B", or "both A and B". In addition, in the present disclosure, the expression "at least one of A or B" or "at least one of A and/or B" may be interpreted as "at least one of A and B".

[0053] In addition, in the present disclosure, "at least one of A, B, and C" may mean "only A", "only B", "only C", or "any combination of A, B, and C". In addition, "at least one of A, B, or C" or "at least one of A, B, and/or C" may mean "at least one of A, B, and C".

[0054] In addition, a parenthesis used in the present disclosure may mean "for example". Specifically, when indicated as "prediction (intra prediction)", it may mean that "intra prediction" is proposed as an example of "prediction". In other words, the "prediction" of the present disclosure is not limited to "intra prediction", and "intra prediction" may be proposed as an example of "prediction". In addition, when indicated as "prediction (i.e., intra prediction)", it may also mean that "intra prediction" is proposed as an example of "prediction".

[0055] Technical features individually described in one figure in the present disclosure may be individually implemented or may be simultaneously implemented.

[0056] FIG. 1 schematically illustrates an example of a video/image coding system to which the present disclosure is applicable.

[0057] Referring to FIG. 1, the video/image coding system may include a first device (source device) and a second device (receive device). The source device may deliver

encoded video/image information or data in the form of a file or streaming to the receive device via a digital storage medium or network.

[0058] The source device may include a video source, an encoding apparatus, and a transmitter. The receive device may include a receiver, a decoding apparatus, and a renderer. The encoding apparatus may be called a video/image encoding apparatus, and the decoding apparatus may be called a video/image decoding apparatus. The transmitter may be included in the encoding apparatus. The receiver may be included in the decoding apparatus. The renderer may include a display, and the display may be configured as a separate device or an external component.

[0059] The video source may obtain a video/image through a process of capturing, synthesizing, or generating a video/image. The video source may include a video/image capture device and/or a video/image generating device. The video/image capture device may include, for example, one or more cameras, video/image archives including previously captured video/images, or the like. The video/image generating device may include, for example, a computer, a tablet and a smartphone, and may (electronically) generate a video/image. For example, a virtual video/image may be generated through a computer or the like. In this case, the video/image capturing process may be replaced by a process of generating related data.

[0060] The encoding apparatus may encode an input video/image. The encoding apparatus may perform a series of procedures such as prediction, transform, and quantization for compression and coding efficiency. The encoded data (encoded video/image information) may be output in the form of a bitstream.

[0061] The transmitter may transmit the encoded video/image information or data output in the form of a bitstream to the receiver of the receive device through a digital storage medium or a network in the form of a file or streaming. The digital storage medium may include various storage mediums such as USB, SD, CD, DVD, Blu-ray, HDD, SSD, and the like. The transmitter may include an element for generating a media file through a predetermined file format, and may include an element for transmission through a broadcast/communication network. The receiver may receive/extract the bitstream and transmit the received/extracted bitstream to the decoding apparatus.

[0062] The decoding apparatus may decode a video/image by performing a series of procedures such as dequantization, inverse transform, prediction, and the like corresponding to the operation of the encoding apparatus.

[0063] The renderer may render the decoded video/image. The rendered video/image may be displayed through the display.

[0064] FIG. 2 is a diagram schematically illustrating a configuration of a video/image encoding apparatus to which the present disclosure is applicable. Hereinafter, what is referred to as the video encoding apparatus may include an image encoding apparatus.

[0065] Referring to FIG. 2, the encoding apparatus 200 may include an image partitioner 210, a predictor 220, a residual processor 230, an entropy encoder 240, an adder 250, a filter 260, and a memory 270. The predictor 220 may include an inter predictor 221 and an intra predictor 222. The residual processor 230 may include a transformer 232, a quantizer 233, a dequantizer 234, an inverse transformer 235. The residual processor 230 may further include a

subtractor **231**. The adder **250** may be called a reconstructor or reconstructed block generator. The image partitioner **210**, the predictor **220**, the residual processor **230**, the entropy encoder **240**, the adder **250**, and the filter **260**, which have been described above, may be constituted by one or more hardware components (e.g., encoder chipsets or processors) according to an embodiment. Further, the memory **270** may include a decoded picture buffer (DPB), and may be constituted by a digital storage medium. The hardware component may further include the memory **270** as an internal/external component.

[0066] The image partitioner **210** may partition an input image (or a picture or a frame) input to the encoding apparatus **200** into one or more processing units. As one example, the processing unit may be called a coding unit (CU). In this case, starting with a coding tree unit (CTU) or the largest coding unit (LCU), the coding unit may be recursively partitioned according to the Quad-tree binary-tree ternary-tree (QTBT) structure. For example, one coding unit may be divided into a plurality of coding units of a deeper depth based on the quad-tree structure, the binary-tree structure, and/or the ternary structure. In this case, for example, the quad-tree structure may be applied first and the binary-tree structure and/or the ternary structure may be applied later. Alternatively, the binary-tree structure may be applied first. The coding procedure according to the present disclosure may be performed based on the final coding unit which is not further partitioned. In this case, the maximum coding unit may be used directly as a final coding unit based on coding efficiency according to the image characteristic.

[0067] Alternatively, the coding unit may be recursively partitioned into coding units of a further deeper depth as needed, so that the coding unit of an optimal size may be used as a final coding unit. Here, the coding procedure may include procedures such as prediction, transform, and reconstruction, which will be described later. As another example, the processing unit may further include a prediction unit (PU) or a transform unit (TU). In this case, the prediction unit and the transform unit may be split or partitioned from the above-described final coding unit. The prediction unit may be a unit of sample prediction, and the transform unit may be a unit for deriving a transform coefficient and/or a unit for deriving a residual signal from a transform coefficient.

[0068] The unit and a term such as a block, an area, or the like may be used in place of each other according to circumstances. In a general case, an M×N block may represent a set of samples or transform coefficients consisting of M columns and N rows. The sample may generally represent a pixel or a value of a pixel, and may represent only a pixel/pixel value of a luma component, or only a pixel/pixel value of a chroma component. The sample may be used as a term corresponding to a pixel or a pel of one picture (or image).

[0069] The subtractor **231** subtracts a prediction signal (predicted block, prediction sample array) output from the predictor **220** from an input image signal (original block, original sample array) to generate a residual signal (residual block, residual sample array), and the generated residual signal is transmitted to the transformer **232**. The predictor **220** may perform prediction on a processing target block (hereinafter, referred to as 'current block'), and may generate a predicted block including prediction samples for the

current block. The predictor **220** may determine whether intra prediction or inter prediction is applied on a current block or CU basis. As discussed later in the description of each prediction mode, the predictor may generate various information relating to prediction, such as prediction mode information, and transmit the generated information to the entropy encoder **240**. The information on the prediction may be encoded in the entropy encoder **240** and output in the form of a bitstream.

[0070] The intra predictor **222** may predict the current block by referring to samples in the current picture. The referred samples may be located in the neighbor of or apart from the current block according to the prediction mode. In the intra prediction, prediction modes may include a plurality of non-directional modes and a plurality of directional modes. The non-directional modes may include, for example, a DC mode and a planar mode. The directional mode may include, for example, 33 directional prediction modes or 65 directional prediction modes according to the degree of detail of the prediction direction. However, this is merely an example, and more or less directional prediction modes may be used depending on a setting. The intra predictor **222** may determine the prediction mode applied to the current block by using the prediction mode applied to the neighboring block.

[0071] The inter predictor **221** may derive a predicted block for the current block based on a reference block (reference sample array) specified by a motion vector on a reference picture. At this time, in order to reduce the amount of motion information transmitted in the inter prediction mode, the motion information may be predicted on a block, subblock, or sample basis based on correlation of motion information between the neighboring block and the current block. The motion information may include a motion vector and a reference picture index. The motion information may further include inter prediction direction (L0 prediction, L1 prediction, Bi prediction, etc.) information. In the case of inter prediction, the neighboring block may include a spatial neighboring block existing in the current picture and a temporal neighboring block existing in the reference picture. The reference picture including the reference block and the reference picture including the temporal neighboring block may be same to each other or different from each other. The temporal neighboring block may be called a collocated reference block, a collocated CU (colCU), and the like, and the reference picture including the temporal neighboring block may be called a collocated picture (colPic). For example, the inter predictor **221** may configure a motion information candidate list based on neighboring blocks and generate information indicating which candidate is used to derive a motion vector and/or a reference picture index of the current block. Inter prediction may be performed based on various prediction modes. For example, in the case of a skip mode and a merge mode, the inter predictor **221** may use motion information of the neighboring block as motion information of the current block. In the skip mode, unlike the merge mode, the residual signal may not be transmitted. In the case of the motion information prediction (motion vector prediction, MVP) mode, the motion vector of the neighboring block may be used as a motion vector predictor and the motion vector of the current block may be indicated by signaling a motion vector difference.

[0072] The predictor **220** may generate a prediction signal based on various prediction methods. For example, the

predictor may apply intra prediction or inter prediction for prediction on one block, and, as well, may apply intra prediction and inter prediction at the same time. This may be called combined inter and intra prediction (CIIP). Further, the predictor may be based on an intra block copy (IBC) prediction mode, or a palette mode in order to perform prediction on a block. The IBC prediction mode or palette mode may be used for content image/video coding of a game or the like, such as screen content coding (SCC). Although the IBC basically performs prediction in a current block, it can be performed similarly to inter prediction in that it derives a reference block in a current block. That is, the IBC may use at least one of inter prediction techniques described in the present disclosure.

[0073] The prediction signal generated through the inter predictor 221 and/or the intra predictor 222 may be used to generate a reconstructed signal or to generate a residual signal. The transformer 232 may generate transform coefficients by applying a transform technique to the residual signal. For example, the transform technique may include at least one of a discrete cosine transform (DCT), a discrete sine transform (DST), a Karhunen-Loève transform (KLT), a graph-based transform (GBT), or a conditionally non-linear transform (CNT). Here, the GBT means transform obtained from a graph when relationship information between pixels is represented by the graph. The CNT refers to transform obtained based on a prediction signal generated using all previously reconstructed pixels. In addition, the transform process may be applied to square pixel blocks having the same size or may be applied to blocks having a variable size rather than the square one.

[0074] The quantizer 233 may quantize the transform coefficients and transmit them to the entropy encoder 240, and the entropy encoder 240 may encode the quantized signal (information on the quantized transform coefficients) and output the encoded signal in a bitstream. The information on the quantized transform coefficients may be referred to as residual information. The quantizer 233 may rearrange block type quantized transform coefficients into a one-dimensional vector form based on a coefficient scan order, and generate information on the quantized transform coefficients based on the quantized transform coefficients of the one-dimensional vector form. The entropy encoder 240 may perform various encoding methods such as, for example, exponential Golomb, context-adaptive variable length coding (CAVLC), context-adaptive binary arithmetic coding (CABAC), and the like. The entropy encoder 240 may encode information necessary for video/image reconstruction other than quantized transform coefficients (e.g. values of syntax elements, etc.) together or separately. Encoded information (e.g., encoded video/image information) may be transmitted or stored on a unit basis of a network abstraction layer (NAL) in the form of a bitstream. The video/image information may further include information on various parameter sets such as an adaptation parameter set (APS), a picture parameter set (PPS), a sequence parameter set (SPS), a video parameter set (VPS) or the like. Further, the video/image information may further include general constraint information. In the present disclosure, information and/or syntax elements which are transmitted/signaled to the decoding apparatus from the encoding apparatus may be included in video/image information. The video/image information may be encoded through the above-described encoding procedure and included in the bitstream. The

bitstream may be transmitted through a network, or stored in a digital storage medium. Here, the network may include a broadcast network, a communication network and/or the like, and the digital storage medium may include various storage media such as USB, SD, CD, DVD, Blu-ray, HDD, SSD, and the like. A transmitter (not shown) which transmits a signal output from the entropy encoder 240 and/or a storage (not shown) which stores it may be configured as an internal/external element of the encoding apparatus 200, or the transmitter may be included in the entropy encoder 240.

[0075] Quantized transform coefficients output from the quantizer 233 may be used to generate a prediction signal. For example, by applying dequantization and inverse transform to quantized transform coefficients through the dequantizer 234 and the inverse transformer 235, the residual signal (residual block or residual samples) may be reconstructed. The adder 155 adds the reconstructed residual signal to a prediction signal output from the inter predictor 221 or the intra predictor 222, so that a reconstructed signal (reconstructed picture, reconstructed block, reconstructed sample array) may be generated. When there is no residual for a processing target block as in a case where the skip mode is applied, the predicted block may be used as a reconstructed block. The adder 250 may be called a reconstructor or a reconstructed block generator. The generated reconstructed signal may be used for intra prediction of a next processing target block in the current block, and as described later, may be used for inter prediction of a next picture through filtering.

[0076] Meanwhile, in the picture encoding and/or reconstructing process, luma mapping with chroma scaling (LMCS) may be applied.

[0077] The filter 260 may improve subjective/objective video quality by applying the filtering to the reconstructed signal. For example, the filter 260 may generate a modified reconstructed picture by applying various filtering methods to the reconstructed picture, and may store the modified reconstructed picture in the memory 270, specifically in the DPB of the memory 270. The various filtering methods may include, for example, deblocking filtering, sample adaptive offset, an adaptive loop filter, a bilateral filter or the like. As discussed later in the description of each filtering method, the filter 260 may generate various information relating to filtering, and transmit the generated information to the entropy encoder 240. The information on the filtering may be encoded in the entropy encoder 240 and output in the form of a bitstream.

[0078] The modified reconstructed picture which has been transmitted to the memory 270 may be used as a reference picture in the inter predictor 221. Through this, the encoding apparatus can avoid prediction mismatch in the encoding apparatus 100 and a decoding apparatus when the inter prediction is applied, and can also improve coding efficiency.

[0079] The memory 270 DPB may store the modified reconstructed picture in order to use it as a reference picture in the inter predictor 221. The memory 270 may store motion information of a block in the current picture, from which motion information has been derived (or encoded) and/or motion information of blocks in an already reconstructed picture. The stored motion information may be transmitted to the inter predictor 221 to be utilized as motion information of a neighboring block or motion information of a temporal neighboring block. The memory 270 may store

reconstructed samples of reconstructed blocks in the current picture, and transmit them to the intra predictor 222.

[0080] FIG. 3 is a diagram schematically illustrating a configuration of a video/image decoding apparatus to which the present disclosure is applicable.

[0081] Referring to FIG. 3, the video decoding apparatus 300 may include an entropy decoder 310, a residual processor 320, a predictor 330, an adder 340, a filter 350 and a memory 360. The predictor 330 may include an inter predictor 331 and an intra predictor 332. The residual processor 320 may include a dequantizer 321 and an inverse transformer 322. The entropy decoder 310, the residual processor 320, the predictor 330, the adder 340, and the filter 350, which have been described above, may be constituted by one or more hardware components (e.g., decoder chipsets or processors) according to an embodiment. Further, the memory 360 may include a decoded picture buffer (DPB), and may be constituted by a digital storage medium. The hardware component may further include the memory 360 as an internal/external component.

[0082] When a bitstream including video/image information is input, the decoding apparatus 300 may reconstruct an image correspondingly to a process by which video/image information has been processed in the encoding apparatus of FIG. 2. For example, the decoding apparatus 300 may derive units/blocks based on information relating to block partition obtained from the bitstream. The decoding apparatus 300 may perform decoding by using a processing unit applied in the encoding apparatus. Therefore, the processing unit of decoding may be, for example, a coding unit, which may be partitioned along the quad-tree structure, the binary-tree structure, and/or the ternary-tree structure from a coding tree unit or a largest coding unit. One or more transform units may be derived from the coding unit. And, the reconstructed image signal decoded and output through the decoding apparatus 300 may be reproduced through a reproducer.

[0083] The decoding apparatus 300 may receive a signal output from the encoding apparatus of FIG. 2 in the form of a bitstream, and the received signal may be decoded through the entropy decoder 310. For example, the entropy decoder 310 may parse the bitstream to derive information (e.g., video/image information) required for image reconstruction (or picture reconstruction). The video/image information may further include information on various parameter sets such as an adaptation parameter set (APS), a picture parameter set (PPS), a sequence parameter set (SPS), a video parameter set (VPS) or the like. Further, the video/image information may further include general constraint information. The decoding apparatus may decode a picture further based on information on the parameter set and/or the general constraint information. In the present disclosure, signaled/received information and/or syntax elements, which will be described later, may be decoded through the decoding procedure and be obtained from the bitstream. For example, the entropy decoder 310 may decode information in the bitstream based on a coding method such as exponential Golomb encoding, CAVLC, CABAC, or the like, and may output a value of a syntax element necessary for image reconstruction and quantized values of a transform coefficient regarding a residual. More specifically, a CABAC entropy decoding method may receive a bin corresponding to each syntax element in a bitstream, determine a context model using decoding target syntax element information and decoding information of neighboring and decoding target

blocks, or information of symbol/bin decoded in a previous step, predict bin generation probability according to the determined context model and perform arithmetic decoding of the bin to generate a symbol corresponding to each syntax element value. Here, the CABAC entropy decoding method may update the context model using information of a symbol/bin decoded for a context model of the next symbol/bin after determination of the context model. Information on prediction among information decoded in the entropy decoder 310 may be provided to the predictor (inter predictor 332 and intra predictor 331), and residual values, that is, quantized transform coefficients, on which entropy decoding has been performed in the entropy decoder 310, and associated parameter information may be input to the residual processor 320. The residual processor 320 may derive a residual signal (residual block, residual samples, residual sample array). Further, information on filtering among information decoded in the entropy decoder 310 may be provided to the filter 350. Meanwhile, a receiver (not shown) which receives a signal output from the encoding apparatus may further constitute the decoding apparatus 300 as an internal/external element, and the receiver may be a component of the entropy decoder 310. Meanwhile, the decoding apparatus according to the present disclosure may be called a video/image/picture coding apparatus, and the decoding apparatus may be classified into an information decoder (video/image/picture information decoder) and a sample decoder (video/image/picture sample decoder). The information decoder may include the entropy decoder 310, and the sample decoder may include at least one of the dequantizer 321, the inverse transformer 322, the adder 340, the filter 350, the memory 360, the inter predictor 332, and the intra predictor 331.

[0084] The dequantizer 321 may output transform coefficients by dequantizing the quantized transform coefficients. The dequantizer 321 may rearrange the quantized transform coefficients in the form of a two-dimensional block. In this case, the rearrangement may perform rearrangement based on an order of coefficient scanning which has been performed in the encoding apparatus. The dequantizer 321 may perform dequantization on the quantized transform coefficients using quantization parameter (e.g., quantization step size information), and obtain transform coefficients.

[0085] The dequantizer 322 obtains a residual signal (residual block, residual sample array) by inverse transforming transform coefficients.

[0086] The predictor may perform prediction on the current block, and generate a predicted block including prediction samples for the current block. The predictor may determine whether intra prediction or inter prediction is applied to the current block based on the information on prediction output from the entropy decoder 310, and specifically may determine an intra/inter prediction mode.

[0087] The predictor may generate a prediction signal based on various prediction methods. For example, the predictor may apply intra prediction or inter prediction for prediction on one block, and, as well, may apply intra prediction and inter prediction at the same time. This may be called combined inter and intra prediction (CIIP). In addition, the predictor may perform intra block copy (IBC) for prediction on a block. The intra block copy may be used for content image/video coding of a game or the like, such as screen content coding (SCC). Although the IBC basically performs prediction in a current block, it can be performed

similarly to inter prediction in that it derives a reference block in a current block. That is, the IBC may use at least one of inter prediction techniques described in the present disclosure.

[0088] The intra predictor **331** may predict the current block by referring to the samples in the current picture. The referred samples may be located in the neighbor of or apart from the current block according to the prediction mode. In the intra prediction, prediction modes may include a plurality of non-directional modes and a plurality of directional modes. The intra predictor **331** may determine the prediction mode applied to the current block by using the prediction mode applied to the neighboring block.

[0089] The inter predictor **332** may derive a predicted block for the current block based on a reference block (reference sample array) specified by a motion vector on a reference picture. At this time, in order to reduce the amount of motion information transmitted in the inter prediction mode, the motion information may be predicted on a block, subblock, or sample basis based on correlation of motion information between the neighboring block and the current block. The motion information may include a motion vector and a reference picture index. The motion information may further include inter prediction direction (LO prediction, L1 prediction, Bi prediction, etc.) information. In the case of inter prediction, the neighboring block may include a spatial neighboring block existing in the current picture and a temporal neighboring block existing in the reference picture. For example, the inter predictor **332** may configure a motion information candidate list based on neighboring blocks, and derive a motion vector and/or a reference picture index of the current block based on received candidate selection information. Inter prediction may be performed based on various prediction modes, and the information on prediction may include information indicating a mode of inter prediction for the current block.

[0090] The adder **340** may generate a reconstructed signal (reconstructed picture, reconstructed block, reconstructed sample array) by adding the obtained residual signal to the prediction signal (predicted block, prediction sample array) output from the predictor **330**. When there is no residual for a processing target block as in a case where the skip mode is applied, the predicted block may be used as a reconstructed block.

[0091] The adder **340** may be called a reconstructor or a reconstructed block generator. The generated reconstructed signal may be used for intra prediction of a next processing target block in the current block, and as described later, may be output through filtering or be used for inter prediction of a next picture.

[0092] Meanwhile, in the picture decoding process, luma mapping with chroma scaling (LMCS) may be applied.

[0093] The filter **350** may improve subjective/objective video quality by applying the filtering to the reconstructed signal. For example, the filter **350** may generate a modified reconstructed picture by applying various filtering methods to the reconstructed picture, and may transmit the modified reconstructed picture in the memory **360**, specifically in the DPB of the memory **360**. The various filtering methods may include, for example, deblocking filtering, sample adaptive offset, an adaptive loop filter, a bilateral filter or the like.

[0094] The (modified) reconstructed picture which has been stored in the DPB of the memory **360** may be used as a reference picture in the inter predictor **332**. The memory

360 may store motion information of a block in the current picture, from which motion information has been derived (or decoded) and/or motion information of blocks in an already reconstructed picture. The stored motion information may be transmitted to the inter predictor **260** to be utilized as motion information of a neighboring block or motion information of a temporal neighboring block. The memory **360** may store reconstructed samples of reconstructed blocks in the current picture, and transmit them to the intra predictor **331**.

[0095] In this specification, the examples described in the predictor **330**, the dequantizer **321**, the inverse transformer **322**, and the filter **350** of the decoding apparatus **300** may be similarly or correspondingly applied to the predictor **220**, the dequantizer **234**, the inverse transformer **235**, and the filter **260** of the encoding apparatus **200**, respectively.

[0096] As described above, prediction is performed in order to increase compression efficiency in performing video coding. Through this, a predicted block including prediction samples for a current block, which is a coding target block, may be generated. Here, the predicted block includes prediction samples in a space domain (or pixel domain). The predicted block may be indentially derived in the encoding apparatus and the decoding apparatus, and the encoding apparatus may increase image coding efficiency by signaling to the decoding apparatus not original sample value of an original block itself but information on residual (residual information) between the original block and the predicted block. The decoding apparatus may derive a residual block including residual samples based on the residual information, generate a reconstructed block including reconstructed samples by adding the residual block to the predicted block, and generate a reconstructed picture including reconstructed blocks.

[0097] The residual information may be generated through transform and quantization procedures. For example, the encoding apparatus may derive a residual block between the original block and the predicted block, derive transform coefficients by performing a transform procedure on residual samples (residual sample array) included in the residual block, and derive quantized transform coefficients by performing a quantization procedure on the transform coefficients, so that it may signal associated residual information to the decoding apparatus (through a bitstream). Here, the residual information may include value information, position information, a transform technique, transform kernel, a quantization parameter or the like of the quantized transform coefficients. The decoding apparatus may perform a quantization/dequantization procedure and derive the residual samples (or residual sample block), based on residual information. The decoding apparatus may generate a reconstructed block based on a predicted block and the residual block. The encoding apparatus may derive a residual block by dequantizing/inverse transforming quantized transform coefficients for reference for inter prediction of a next picture, and may generate a reconstructed picture based on this.

[0098] FIG. 4 schematically illustrates a multiple transform technique according to an embodiment of the present disclosure.

[0099] Referring to FIG. 4, a transformer may correspond to the transformer in the encoding apparatus of foregoing FIG. 2, and an inverse transformer may correspond to the

inverse transformer in the encoding apparatus of foregoing FIG. 2, or to the inverse transformer in the decoding apparatus of FIG. 3.

[0100] The transformer may derive (primary) transform coefficients by performing a primary transform based on residual samples (residual sample array) in a residual block (S410). This primary transform may be referred to as a core transform. Herein, the primary transform may be based on multiple transform selection (MTS), and when a multiple transform is applied as the primary transform, it may be referred to as a multiple core transform.

[0101] The multiple core transform may represent a method of transforming additionally using discrete cosine transform (DCT) type 2 and discrete sine transform (DST) type 7, DCT type 8, and/or DST type 1. That is, the multiple core transform may represent a transform method of transforming a residual signal (or residual block) of a space domain into transform coefficients (or primary transform coefficients) of a frequency domain based on a plurality of transform kernels selected from among the DCT type 2, the DST type 7, the DCT type 8 and the DST type 1. Herein, the primary transform coefficients may be called temporary transform coefficients from the viewpoint of the transformer.

[0102] In other words, when the conventional transform method is applied, transform coefficients might be generated by applying transform from a space domain to a frequency domain for a residual signal (or residual block) based on the DCT type 2. Unlike to this, when the multiple core transform is applied, transform coefficients (or primary transform coefficients) may be generated by applying transform from a space domain to a frequency domain for a residual signal (or residual block) based on the DCT type 2, the DST type 7, the DCT type 8, and/or DST type 1. Herein, the DCT type 2, the DST type 7, the DCT type 8, and the DST type 1 may be called a transform type, transform kernel or transform core. These DCT/DST transform types can be defined based on basis functions.

[0103] When the multiple core transform is performed, a vertical transform kernel and a horizontal transform kernel for a target block may be selected from among the transform kernels, a vertical transform may be performed on the target block based on the vertical transform kernel, and a horizontal transform may be performed on the target block based on the horizontal transform kernel. Here, the horizontal transform may indicate a transform on horizontal components of the target block, and the vertical transform may indicate a transform on vertical components of the target block. The vertical transform kernel/horizontal transform kernel may be adaptively determined based on a prediction mode and/or a transform index for the target block (CU or subblock) including a residual block.

[0104] Further, according to an example, if the primary transform is performed by applying the MTS, a mapping relationship for transform kernels may be set by setting specific basis functions to predetermined values and combining basis functions to be applied in the vertical transform or the horizontal transform. For example, when the horizontal transform kernel is expressed as trTypeHor and the vertical direction transform kernel is expressed as trTypeVer, a trTypeHor or trType Ver value of 0 may be set to DCT2, a trTypeHor or trType Ver value of 1 may be set to DST7, and a trTypeHor or trTypeVer value of 2 may be set to DCT8.

[0105] In this case, MTS index information may be encoded and signaled to the decoding apparatus to indicate any one of a plurality of transform kernel sets. For example, an MTS index of 0 may indicate that both trTypeHor and trTypeVer values are 0, an MTS index of 1 may indicate that both trTypeHor and trType Ver values are 1, an MTS index of 2 may indicate that the trTypeHor value is 2 and the trType Ver value is 1, an MTS index of 3 may indicate that the trTypeHor value is 1 and the trType Ver value is 2, and an MTS index of 4 may indicate that both both trTypeHor and trType Ver values are 2.

[0106] In one example, transform kernel sets according to MTS index information are illustrated in the following table.

TABLE 1

tu_mts_idx[x0][y0]	0	1	2	3	4
trTypeHor	0	1	2	1	2
trTypeVer	0	1	1	2	2

[0107] The transformer may derive modified (secondary) transform coefficients by performing the secondary transform based on the (primary) transform coefficients (S420). The primary transform is a transform from a spatial domain to a frequency domain, and the secondary transform refers to transforming into a more compressive expression by using a correlation existing between (primary) transform coefficients. The secondary transform may include a non-separable transform. In this case, the secondary transform may be called a non-separable secondary transform (NSST), or a mode-dependent non-separable secondary transform (MDNSST). The non-separable secondary transform may represent a transform which generates modified transform coefficients (or secondary transform coefficients) for a residual signal by secondary-transforming, based on a non-separable transform matrix, (primary) transform coefficients derived through the primary transform. At this time, the vertical transform and the horizontal transform may not be applied separately (or horizontal and vertical transforms may not be applied independently) to the (primary) transform coefficients, but the transforms may be applied at once based on the non-separable transform matrix. In other words, the non-separable secondary transform may represent a transform method in which the vertical and horizontal components of the (primary) transform coefficients are not separated, and for example, two-dimensional signals (transform coefficients) are re-arranged to a one-dimensional signal through a certain determined direction (e.g., row-first direction or column-first direction), and then modified transform coefficients (or secondary transform coefficients) are generated based on the non-separable transform matrix. For example, according to a row-first order, MxN blocks are disposed in a line in an order of a first row, a second row, . . . , and an Nth row. According to a column-first order, MxN blocks are disposed in a line in an order of a first column, a second column, . . . , and an Nth column. The non-separable secondary transform may be applied to a top-left region of a block configured with (primary) transform coefficients (hereinafter, may be referred to as a transform coefficient block). For example, if the width (W) and the height (H) of the transform coefficient block are all equal to or greater than 8, an 8x8 non-separable secondary transform may be applied to a top-left 8x8 region of the transform coefficient block. Further, if the width (W) and the height (H) of the transform

coefficient block are all equal to or greater than 4, and the width (W) or the height (H) of the transform coefficient block is less than 8, then a 4×4 non-separable secondary transform may be applied to a top-left min (8,W)×min (8,H) region of the transform coefficient block. However, the embodiment is not limited to this, and for example, even if only the condition that the width (W) or height (H) of the transform coefficient block is equal to or greater than 4 is satisfied, the 4×4 non-separable secondary transform may be applied to the top-left min (8, W)×min (8,H) region of the transform coefficient block.

[0108] Specifically, for example, if a 4×4 input block is used, the non-separable secondary transform may be performed as follows.

[0109] The 4×4 input block X may be represented as follows.

$$X = \begin{bmatrix} X_{00} & X_{01} & X_{02} & X_{03} \\ X_{10} & X_{11} & X_{12} & X_{13} \\ X_{20} & X_{21} & X_{22} & X_{23} \\ X_{30} & X_{31} & X_{32} & X_{33} \end{bmatrix} \quad [\text{Equation 1}]$$

[0110] If the X is represented in the form of a vector, the vector $\vec{X}$ may be represented as below.

$$\vec{X} = [X_{00} \ X_{01} \ X_{02} \ X_{03} \ X_{10} \ X_{11} \ X_{12} \ X_{13} \ X_{20} \ X_{21} \ X_{22} \ X_{23} \ X_{30} \ X_{31} \ X_{32} \ X_{33}]^T \quad [\text{Equation 2}]$$

[0111] In Equation 2, the vector $\vec{X}$ is a one-dimensional vector obtained by rearranging the two-dimensional block X of Equation 1 according to the row-first order.

[0112] In this case, the secondary non-separable transform may be calculated as below.

$$\vec{F} = T \cdot \vec{X} \quad [\text{Equation 3}]$$

[0113] In this equation, $\vec{F}$ represents a transform coefficient vector, and T represents a 16×16 (non-separable) transform matrix.

[0114] Through foregoing Equation 3, a 16×1 transform coefficient vector $\vec{F}$ may be derived, and the $\vec{F}$ may be re-organized into a 4×4 block through a scan order (horizontal, vertical, diagonal and the like). However, the above-described calculation is an example, and hypercube-Givens transform (HyGT) or the like may be used for the calculation of the non-separable secondary transform in order to reduce the computational complexity of the non-separable secondary transform.

[0115] Meanwhile, in the non-separable secondary transform, a transform kernel (or transform core, transform type) may be selected to be mode dependent. In this case, the mode may include the intra prediction mode and/or the inter prediction mode.

[0116] As described above, the non-separable secondary transform may be performed based on an 8×8 transform or a 4×4 transform determined based on the width (W) and the height (H) of the transform coefficient block. The 8×8 transform refers to a transform that is applicable to an 8×8

region included in the transform coefficient block when both W and H are equal to or greater than 8, and the 8×8 region may be a top-left 8×8 region in the transform coefficient block. Similarly, the 4×4 transform refers to a transform that is applicable to a 4×4 region included in the transform coefficient block when both W and H are equal to or greater than 4, and the 4×4 region may be a top-left 4×4 region in the transform coefficient block. For example, an 8×8 transform kernel matrix may be a 64×64/16×64 matrix, and a 4×4 transform kernel matrix may be a 16×16/8×16 matrix.

[0117] Here, to select a mode-dependent transform kernel, two non-separable secondary transform kernels per transform set for a non-separable secondary transform may be configured for both the 8×8 transform and the 4×4 transform, and there may be four transform sets. That is, four transform sets may be configured for the 8×8 transform, and four transform sets may be configured for the 4×4 transform. In this case, each of the four transform sets for the 8×8 transform may include two 8×8 transform kernels, and each of the four transform sets for the 4×4 transform may include two 4×4 transform kernels.

[0118] However, as the size of the transform, that is, the size of a region to which the transform is applied, may be, for example, a size other than 8×8 or 4×4, the number of sets may be n, and the number of transform kernels in each set may be k.

[0119] The transform set may be referred to as an NSST set or an LFNST set. A specific set among the transform sets may be selected, for example, based on the intra prediction mode of the current block (CU or subblock). A low-frequency non-separable transform (LFNST) may be an example of a reduced non-separable transform, which will be described later, and represents a non-separable transform for a low frequency component.

[0120] For reference, for example, the intra prediction mode may include two non-directional (or non-angular) intra prediction modes and 65 directional (or angular) intra prediction modes. The non-directional intra prediction modes may include a planar intra prediction mode of No. 0 and a DC intra prediction mode of No. 1, and the directional intra prediction modes may include 65 intra prediction modes of Nos. 2 to 66. However, this is an example, and this document may be applied even when the number of intra prediction modes is different. Meanwhile, in some cases, intra prediction mode No. 67 may be further used, and the intra prediction mode No. 67 may represent a linear model (LM) mode.

[0121] FIG. 5 exemplarily shows intra directional modes of 65 prediction directions.

[0122] Referring to FIG. 5, on the basis of intra prediction mode 34 having a left upward diagonal prediction direction, the intra prediction modes may be divided into intra prediction modes having horizontal directionality and intra prediction modes having vertical directionality. In FIGS. 5, H and V denote horizontal directionality and vertical directionality, respectively, and numerals -32 to 32 indicate displacements in 1/32 units on a sample grid position. These numerals may represent an offset for a mode index value. Intra prediction modes 2 to 33 have the horizontal direc-

tionality, and intra prediction modes 34 to 66 have the vertical directionality. Strictly speaking, intra prediction mode 34 may be considered as being neither horizontal nor vertical, but may be classified as belonging to the horizontal directionality in determining a transform set of a secondary transform. This is because input data is transposed to be used for a vertical direction mode symmetrical on the basis of intra prediction mode 34, and an input data alignment method for a horizontal mode is used for intra prediction mode 34. Transposing input data means that rows and columns of two-dimensional M×N block data are switched into N×M data. Intra prediction mode 18 and intra prediction mode 50 may represent a horizontal intra prediction mode and a vertical intra prediction mode, respectively, and intra prediction mode 2 may be referred to as a right upward diagonal intra prediction mode because intra prediction mode 2 has a left reference pixel and performs prediction in a right upward direction. Likewise, intra prediction mode 34 may be referred to as a right downward diagonal intra prediction mode, and intra prediction mode 66 may be referred to as a left downward diagonal intra prediction mode.

[0123] According to an example, the four transform sets according to the intra prediction mode may be mapped, for example, as shown in the following table.

TABLE 2

lfstPredModeIntra	lfstTrSetIdx
lfstPredModeIntra < 0	1
0 ≤ lfstPredModeIntra ≤ 1	0
2 ≤ lfstPredModeIntra ≤ 12	1
13 ≤ lfstPredModeIntra ≤ 23	2
24 ≤ lfstPredModeIntra ≤ 44	3
45 ≤ lfstPredModeIntra ≤ 55	2
56 ≤ lfstPredModeIntra ≤ 80	1
81 ≤ lfstPredModeIntra ≤ 83	0

[0124] As shown in Table 2, any one of the four transform sets, that is, lfstTrSetIdx, may be mapped to any one of four indexes, that is, 0 to 3, according to the intra prediction mode.

[0125] When it is determined that a specific set is used for the non-separable transform, one of k transform kernels in the specific set may be selected through a non-separable secondary transform index. An encoding apparatus may derive a non-separable secondary transform index indicating a specific transform kernel based on a rate-distortion (RD) check and may signal the non-separable secondary transform index to a decoding apparatus. The decoding apparatus may select one of the k transform kernels in the specific set based on the non-separable secondary transform index. For example, lfst index value 0 may refer to a first non-separable secondary transform kernel, lfst index value 1 may refer to a second non-separable secondary transform kernel, and lfst index value 2 may refer to a third non-separable secondary transform kernel. Alternatively, lfst index value 0 may indicate that the first non-separable secondary transform is not applied to the target block, and lfst index values 1 to 3 may indicate the three transform kernels.

[0126] The transformer may perform the non-separable secondary transform based on the selected transform kernels, and may obtain modified (secondary) transform coefficients. As described above, the modified transform coef-

ficients may be derived as transform coefficients quantized through the quantizer, and may be encoded and signaled to the decoding apparatus and transferred to the dequantizer/inverse transformer in the encoding apparatus.

[0127] Meanwhile, as described above, if the secondary transform is omitted, (primary) transform coefficients, which are an output of the primary (separable) transform, may be derived as transform coefficients quantized through the quantizer as described above, and may be encoded and signaled to the decoding apparatus and transferred to the dequantizer/inverse transformer in the encoding apparatus.

[0128] The inverse transformer may perform a series of procedures in the inverse order to that in which they have been performed in the above-described transformer. The inverse transformer may receive (dequantized) transformer coefficients, and derive (primary) transform coefficients by performing a secondary (inverse) transform (S450), and may obtain a residual block (residual samples) by performing a primary (inverse) transform on the (primary) transform coefficients (S460). In this connection, the primary transform coefficients may be called modified transform coefficients from the viewpoint of the inverse transformer. As described above, the encoding apparatus and the decoding apparatus may generate the reconstructed block based on the residual block and the predicted block, and may generate the reconstructed picture based on the reconstructed block.

[0129] The decoding apparatus may further include a secondary inverse transform application determinator (or an element to determine whether to apply a secondary inverse transform) and a secondary inverse transform determinator (or an element to determine a secondary inverse transform). The secondary inverse transform application determinator may determine whether to apply a secondary inverse transform. For example, the secondary inverse transform may be an NSST, an RST, or an LFNST and the secondary inverse transform application determinator may determine whether to apply the secondary inverse transform based on a secondary transform flag obtained by parsing the bitstream. In another example, the secondary inverse transform application determinator may determine whether to apply the secondary inverse transform based on a transform coefficient of a residual block.

[0130] The secondary inverse transform determinator may determine a secondary inverse transform. In this case, the secondary inverse transform determinator may determine the secondary inverse transform applied to the current block based on an LFNST (NSST or RST) transform set specified according to an intra prediction mode. In an embodiment, a secondary transform determination method may be determined depending on a primary transform determination method. Various combinations of primary transforms and secondary transforms may be determined according to the intra prediction mode. Further, in an example, the secondary inverse transform determinator may determine a region to which a secondary inverse transform is applied based on the size of the current block.

[0131] Meanwhile, as described above, if the secondary (inverse) transform is omitted, (dequantized) transform coefficients may be received, the primary (separable) inverse transform may be performed, and the residual block (residual samples) may be obtained. As described above, the encoding apparatus and the decoding apparatus may generate the reconstructed block based on the residual block and

the predicted block, and may generate the reconstructed picture based on the reconstructed block.

[0132] Meanwhile, in the present disclosure, a reduced secondary transform (RST) in which the size of a transform matrix (kernel) is reduced may be applied in the concept of NSST in order to reduce the amount of computation and memory required for the non-separable secondary transform.

[0133] Meanwhile, the transform kernel, the transform matrix, and the coefficient constituting the transform kernel matrix, that is, the kernel coefficient or the matrix coefficient, described in the present disclosure may be expressed in 8 bits. This may be a condition for implementation in the decoding apparatus and the encoding apparatus, and may reduce the amount of memory required to store the transform kernel with a performance degradation that can be reasonably accommodated compared to the existing 9 bits or 10 bits. In addition, the expressing of the kernel matrix in 8 bits may allow a small multiplier to be used, and may be more suitable for single instruction multiple data (SIMD) instructions used for optimal software implementation.

[0134] In the present specification, the term “RST” may mean a transform which is performed on residual samples for a target block based on a transform matrix whose size is reduced according to a reduced factor. In the case of performing the reduced transform, the amount of computation required for transform may be reduced due to a reduction in the size of the transform matrix. That is, the RST may be used to address the computational complexity issue occurring at the non-separable transform or the transform of a block of a great size.

[0135] RST may be referred to as various terms, such as reduced transform, reduced secondary transform, reduction transform, simplified transform, simple transform, and the like, and the name which RST may be referred to as is not limited to the listed examples. Alternatively, since the RST is mainly performed in a low frequency region including a non-zero coefficient in a transform block, it may be referred to as a Low-Frequency Non-Separable Transform (LFNST). The transform index may be referred to as an LFNST index.

[0136] Meanwhile, when the secondary inverse transform is performed based on RST, the inverse transformer 235 of the encoding apparatus 200 and the inverse transformer 322 of the decoding apparatus 300 may include an inverse reduced secondary transformer which derives modified transform coefficients based on the inverse RST of the transform coefficients, and an inverse primary transformer which derives residual samples for the target block based on the inverse primary transform of the modified transform coefficients. The inverse primary transform refers to the inverse transform of the primary transform applied to the residual. In the present disclosure, deriving a transform coefficient based on a transform may refer to deriving a transform coefficient by applying the transform.

[0137] FIG. 6 is a diagram illustrating an RST according to an embodiment of the present disclosure.

[0138] In the present disclosure, a “target block” may refer to a current block to be coded, a residual block, or a transform block.

[0139] In the RST according to an example, an N-dimensional vector may be mapped to an R-dimensional vector located in another space, so that the reduced transform matrix may be determined, where R is less than N. N may mean the square of the length of a side of a block to which

the transform is applied, or the total number of transform coefficients corresponding to a block to which the transform is applied, and the reduced factor may mean an R/N value. The reduced factor may be referred to as a reduced factor, reduction factor, simplified factor, simple factor or other various terms. Meanwhile, R may be referred to as a reduced coefficient, but according to circumstances, the reduced factor may mean R. Further, according to circumstances, the reduced factor may mean the N/R value.

[0140] In an example, the reduced factor or the reduced coefficient may be signaled through a bitstream, but the example is not limited to this. For example, a predefined value for the reduced factor or the reduced coefficient may be stored in each of the encoding apparatus 200 and the decoding apparatus 300, and in this case, the reduced factor or the reduced coefficient may not be signaled separately.

[0141] The size of the reduced transform matrix according to an example may be R×N less than N×N, the size of a conventional transform matrix, and may be defined as in Equation 4 below.

$$T_{R \times N} = \begin{bmatrix} t_{11} & t_{12} & t_{13} & \dots & t_{1N} \\ t_{21} & t_{22} & t_{23} & & t_{2N} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ t_{R1} & t_{R2} & t_{R3} & \dots & t_{RN} \end{bmatrix} \quad [\text{Equation 4}]$$

[0142] The matrix T in the Reduced Transform block shown in FIG. 6(a) may mean the matrix $T_{R \times N}$ of Equation 4. As shown in FIG. 6(a), when the reduced transform matrix $T_{R \times N}$ is multiplied to residual samples for the target block, transform coefficients for the target block may be derived.

[0143] In an example, if the size of the block to which the transform is applied is 8×8 and R=16 (i.e., R/N=16/64=1/4), then the RST according to FIG. 6(a) may be expressed as a matrix operation as shown in Equation 5 below. In this case, memory and multiplication calculation can be reduced to approximately 1/4 by the reduced factor.

[0144] In the present disclosure, a matrix operation may be understood as an operation of multiplying a column vector by a matrix, disposed on the left of the column vector, to obtain a column vector.

$$\begin{bmatrix} t_{1,1} & t_{1,2} & t_{1,3} & \dots & t_{1,64} \\ t_{2,1} & t_{2,2} & t_{2,3} & & t_{2,64} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ t_{16,1} & t_{16,2} & t_{16,3} & \dots & t_{16,64} \end{bmatrix} \times \begin{bmatrix} r_1 \\ r_2 \\ \vdots \\ r_{64} \end{bmatrix} \quad [\text{Equation 5}]$$

[0145] In Equation 5, r_1 to r_{64} may represent residual samples for the target block and may be specifically transform coefficients generated by applying a primary transform. As a result of the calculation of Equation 5 transform coefficients ci for the target block may be derived, and a process of deriving ci may be as in Equation 6.

```

for i from 0 to R:
  ci = 0
  for j from 1 to N
    ci += tij * rj

```

[Equation 6]

[0146] As a result of the calculation of Equation 6, transform coefficients c_1 to c_R for the target block may be derived. That is, when $R=16$, transform coefficients c_1 to c_{16} for the target block may be derived. If, instead of RST, a regular transform is applied and a transform matrix of 64×64 ($N \times N$) size is multiplied to residual samples of 64×1 ($N \times 1$) size, then only 16 (R) transform coefficients are derived for the target block because RST was applied, although 64 (N) transform coefficients are derived for the target block. Since the total number of transform coefficients for the target block is reduced from N to R , the amount of data transmitted by the encoding apparatus 200 to the decoding apparatus 300 decreases, so efficiency of transmission between the encoding apparatus 200 and the decoding apparatus 300 can be improved.

[0147] When considered from the viewpoint of the size of the transform matrix, the size of the regular transform matrix is 64×64 ($N \times N$), but the size of the reduced transform matrix is reduced to 16×64 ($R \times N$), so memory usage in a case of performing the RST can be reduced by an R/N ratio when compared with a case of performing the regular transform. In addition, when compared to the number of multiplication calculations $N \times N$ in a case of using the regular transform matrix, the use of the reduced transform matrix can reduce the number of multiplication calculations by the R/N ratio ($R \times N$).

[0148] In an example, the transformer 232 of the encoding apparatus 200 may derive transform coefficients for the target block by performing the primary transform and the RST-based secondary transform on residual samples for the target block. These transform coefficients may be transferred to the inverse transformer of the decoding apparatus 300, and the inverse transformer 322 of the decoding apparatus 300 may derive the modified transform coefficients based on the inverse reduced secondary transform (RST) for the transform coefficients, and may derive residual samples for the target block based on the inverse primary transform for the modified transform coefficients.

[0149] The size of the inverse RST matrix $T_{N \times R}$ according to an example is $N \times R$ less than the size $N \times N$ of the regular inverse transform matrix, and is in a transpose relationship with the reduced transform matrix $T_{R \times N}$ shown in Equation 4.

[0150] The matrix Tt in the Reduced Inv. Transform block shown in FIG. 6(b) may mean the inverse RST matrix $T_{R \times N}^T$ (the superscript T means transpose). When the inverse RST matrix $T_{R \times N}^T$ is multiplied to the transform coefficients for the target block as shown in FIG. 6(b), the modified transform coefficients for the target block or the residual samples for the current block may be derived. The inverse RST matrix $T_{R \times N}^T$ may be expressed as $(T_{R \times N})^T N \times R$.

[0151] More specifically, when the inverse RST is applied as the secondary inverse transform, the modified transform coefficients for the target block may be derived when the inverse RST matrix $T_{R \times N}^T$ is multiplied to the transform coefficients for the target block. Meanwhile, the inverse RST may be applied as the inverse primary transform, and in this case, the residual samples for the target block may be derived when the inverse RST matrix $T_{R \times N}^T$ is multiplied to the transform coefficients for the target block.

[0152] In an example, if the size of the block to which the inverse transform is applied is 8×8 and $R=16$ (i.e., $R/N=16/$

$64=1/4$), then the RST according to FIG. 6(b) may be expressed as a matrix operation as shown in Equation 7 below.

$$\begin{bmatrix} t_{1,1} & t_{2,1} & & t_{16,1} \\ t_{1,2} & t_{2,2} & \dots & t_{16,2} \\ t_{1,3} & t_{2,3} & & t_{16,3} \\ \vdots & \vdots & & \vdots \\ \vdots & \vdots & \ddots & \vdots \\ t_{1,64} & t_{2,64} & \dots & t_{16,64} \end{bmatrix} \times \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_{16} \end{bmatrix} \quad [\text{Equation 7}]$$

[0153] In Equation 7, c_1 to c_{16} may represent the transform coefficients for the target block. As a result of the calculation of Equation 7, r_i representing the modified transform coefficients for the target block or the residual samples for the target block may be derived, and the process of deriving r_i may be as in Equation 8.

$$\begin{array}{l} \text{For } i \text{ from } 1 \text{ to } N \\ r_i = 0 \\ \quad \text{for } j \text{ from } 1 \text{ to } R \\ \quad \quad r_i += t_{ji} * c_j \end{array} \quad [\text{Equation 8}]$$

[0154] As a result of the calculation of Equation 8, r_i to IN representing the modified transform coefficients for the target block or the residual samples for the target block may be derived. When considered from the viewpoint of the size of the inverse transform matrix, the size of the regular inverse transform matrix is 64×64 ($N \times N$), but the size of the reduced inverse transform matrix is reduced to 64×16 ($R \times N$), so memory usage in a case of performing the inverse RST can be reduced by an R/N ratio when compared with a case of performing the regular inverse transform. In addition, when compared to the number of multiplication calculations $N \times N$ in a case of using the regular inverse transform matrix, the use of the reduced inverse transform matrix can reduce the number of multiplication calculations by the R/N ratio ($N \times R$).

[0155] A transform set configuration shown in Table 2 may also be applied to an 8×8 RST. That is, the 8×8 RST may be applied according to a transform set in Table 2. Since one transform set includes two or three transforms (kernels) according to an intra prediction mode, it may be configured to select one of up to four transforms including that in a case where no secondary transform is applied. In a transform where no secondary transform is applied, it may be considered to apply an identity matrix. Assuming that indexes 0, 1, 2, and 3 are respectively assigned to the four transforms (e.g., index 0 may be allocated to a case where an identity matrix is applied, that is, a case where no secondary transform is applied), a transform index or an lfnst index as a syntax element may be signaled for each transform coefficient block, thereby designating a transform to be applied. That is, for a top-left 8×8 block, through the transform index, it is possible to designate an 8×8 RST in an RST configuration, or to designate an 8×8 lfnst when the LFNST is applied. The 8×8 lfnst and the 8×8 RST refer to transforms applicable to an 8×8 region included in the transform coefficient block when both W and H of the target block to be transformed are equal to or greater than 8, and the 8×8 region may be a top-left 8×8 region in the transform coef-

ficient block. Similarly, a 4×4 lfnst and a 4×4 RST refer to transforms applicable to a 4×4 region included in the transform coefficient block when both W and H of the target block to be equal to or greater than 4, and the 4×4 region may be a top-left 4×4 region in the transform coefficient block.

[0156] According to an embodiment of the present disclosure, for a transform in an encoding process, only 48 pieces of data may be selected and a maximum 16×48 transform kernel matrix may be applied thereto, rather than applying a 16×64 transform kernel matrix to 64 pieces of data forming an 8×8 region. Here, “maximum” means that m has a maximum value of 16 in an m×48 transform kernel matrix for generating m coefficients. That is, when an RST is performed by applying an m×48 transform kernel matrix (m≤16) to an 8×8 region, 48 pieces of data are input and m coefficients are generated. When m is 16, 48 pieces of data are input and 16 coefficients are generated. That is, assuming that 48 pieces of data form a 48×1 vector, a 16×48 matrix and a 48×1 vector are sequentially multiplied, thereby generating a 16×1 vector. Here, the 48 pieces of data forming the 8×8 region may be properly arranged, thereby forming the 48×1 vector. For example, a 48×1 vector may be constructed based on 48 pieces of data constituting a region excluding the bottom right 4×4 region among the 8×8 regions. Here, when a matrix operation is performed by applying a maximum 16×48 transform kernel matrix, 16 modified transform coefficients are generated, and the 16 modified transform coefficients may be arranged in a top-left 4×4 region according to a scanning order, and a top-right 4×4 region and a bottom-left 4×4 region may be filled with zeros.

[0157] For an inverse transform in a decoding process, the transposed matrix of the foregoing transform kernel matrix may be used. That is, when an inverse RST or LFNST is performed in the inverse transform process performed by the decoding apparatus, input coefficient data to which the inverse RST is applied is configured in a one-dimensional vector according to a predetermined arrangement order, and a modified coefficient vector obtained by multiplying the one-dimensional vector and a corresponding inverse RST matrix on the left of the one-dimensional vector may be arranged in a two-dimensional block according to a predetermined arrangement order.

[0158] In summary, in the transform process, when an RST or LFNST is applied to an 8×8 region, a matrix operation of 48 transform coefficients in top-left, top-right, and bottom-left regions of the 8×8 region excluding the bottom-right region among transform coefficients in the 8×8 region and a 16×48 transform kernel matrix. For the matrix operation, the 48 transform coefficients are input in a one-dimensional array. When the matrix operation is performed, 16 modified transform coefficients are derived, and the modified transform coefficients may be arranged in the top-left region of the 8×8 region.

[0159] On the contrary, in the inverse transform process, when an inverse RST or LFNST is applied to an 8×8 region, 16 transform coefficients corresponding to a top-left region of the 8×8 region among transform coefficients in the 8×8 region may be input in a one-dimensional array according to a scanning order and may be subjected to a matrix operation with a 48×16 transform kernel matrix. That is, the matrix operation may be expressed as (48×16 matrix)×(16×1 transform coefficient vector)=(48×1 modified transform coefficient

vector). Here, an n×1 vector may be interpreted to have the same meaning as an n×1 matrix and may thus be expressed as an n×1 column vector. Further, * denotes matrix multiplication. When the matrix operation is performed, 48 modified transform coefficients may be derived, and the 48 modified transform coefficients may be arranged in top-left, top-right, and bottom-left regions of the 8×8 region excluding a bottom-right region.

[0160] When a secondary inverse transform is based on an RST, the inverse transformer 235 of the encoding apparatus 200 and the inverse transformer 322 of the decoding apparatus 300 may include an inverse reduced secondary transformer to derive modified transform coefficients based on an inverse RST on transform coefficients and an inverse primary transformer to derive residual samples for the target block based on an inverse primary transform on the modified transform coefficients. The inverse primary transform refers to the inverse transform of a primary transform applied to a residual. In the present disclosure, deriving a transform coefficient based on a transform may refer to deriving the transform coefficient by applying the transform.

[0161] The above-described non-separable transform, the LFNST, will be described in detail as follows. The LFNST may include a forward transform by the encoding apparatus and an inverse transform by the decoding apparatus.

[0162] The encoding apparatus receives a result (or a part of a result) derived after applying a primary (core) transform as an input, and applies a forward secondary transform (secondary transform).

$$y = G^T x$$

[Equation 9]

[0163] In Equation 9, x and y are inputs and outputs of the secondary transform, respectively, and G is a matrix representing the secondary transform, and transform basis vectors are composed of column vectors. In the case of an inverse LFNST, when the dimension of the transformation matrix G is expressed as [number of rows x number of columns], in the case of a forward LFNST, the transposition of matrix G becomes the dimension of GT.

[0164] For the inverse LFNST, the dimensions of matrix G are [48×16], [48×8], [16×48], [16×8], and the [48×8] matrix and the [16×8] matrix are partial matrices that sampled 8 transform basis vectors from the left of the [48×16] matrix and the [16×16] matrix, respectively.

[0165] On the other hand, for the forward LFNST, the dimensions of matrix GT are [16×48], ×48], [16×16], [8×16], and the [8×48] matrix and the [8×16] matrix are partial matrices obtained by sampling 8 transform basis vectors from the top of the [16×48] matrix and the [16×16] matrix, respectively.

[0166] Therefore, in the case of the forward LFNST, a [48×1] vector or [16×1] vector is possible as an input x, and a [16×1] vector or a [8×1] vector is possible as an output y. In video coding and decoding, the output of the forward primary transform is two-dimensional (2D) data, so to construct the [48×1] vector or the [16×1] vector as the input x, a one-dimensional vector must be constructed by properly arranging the 2D data that is the output of the forward transformation.

[0167] FIG. 7 is a diagram illustrating a sequence of arranging output data of a forward primary transformation

into a one-dimensional vector according to an example. The left diagrams of (a) and (b) of FIG. 7 show the sequence for constructing a $[48 \times 1]$ vector, and the right diagrams of (a) and (b) of FIG. 7 shows the sequence for constructing a $[16 \times 1]$ vector. In the case of the LFNST, a one-dimensional vector x can be obtained by sequentially arranging 2D data in the same order as in (a) and (b) of FIG. 7.

[0168] The arrangement direction of the output data of the forward primary transform may be determined according to an intra prediction mode of the current block. For example, when the intra prediction mode of the current block is in the horizontal direction with respect to the diagonal direction, the output data of the forward primary transform may be arranged in the order of (a) of FIG. 7, and when the intra prediction mode of the current block is in the vertical direction with respect to the diagonal direction, the output data of the forward primary transform may be arranged in the order of (b) of FIG. 7.

[0169] According to an example, an arrangement order different from the arrangement orders of (a) and (b) FIG. 7 may be applied, and in order to derive the same result (y vector) as when the arrangement orders of (a) and (b) FIG. 7 is applied, the column vectors of the matrix G may be rearranged according to the arrangement order. That is, it is possible to rearrange the column vectors of G so that each element constituting the x vector is always multiplied by the same transform basis vector.

[0170] Since the output y derived through Equation 9 is a one-dimensional vector, when two-dimensional data is required as input data in the process of using the result of the forward secondary transformation as an input, for example, in the process of performing quantization or residual coding, the output y vector of Equation 9 must be properly arranged as 2D data again.

[0171] FIG. 8 is a diagram illustrating a sequence of arranging output data of a forward secondary transform into a two-dimensional vector according to an example.

[0172] In the case of the LFNST, output values may be arranged in a 2D block according to a predetermined scan order. (a) of FIG. 8 shows that when the output y is a $[16 \times 1]$ vector, the output values are arranged at 16 positions of the 2D block according to a diagonal scan order. (b) of FIG. 8 shows that when the output y is a $[8 \times 1]$ vector, the output values are arranged at 8 positions of the 2D block according to the diagonal scan order, and the remaining 8 positions are filled with zeros. X in (b) of FIG. 8 indicates that it is filled with zero.

[0173] According to another example, since the order in which the output vector y is processed in performing quantization or residual coding may be preset, the output vector y may not be arranged in the 2D block as shown in FIG. 8. However, in the case of the residual coding, data coding may be performed in 2D block (eg, 4×4) units such as CG (Coefficient Group), and in this case, the data are arranged according to a specific order as in the diagonal scan order of FIG. 8.

[0174] Meanwhile, the decoding apparatus may configure the one-dimensional input vector y by arranging two-dimensional data output through a dequantization process or the like according to a preset scan order for the inverse transformation. The input vector y may be output as the output vector x by the following equation.

$$x = Gy$$

[Equation 10]

[0175] In the case of the inverse LFNST, an output vector x can be derived by multiplying an input vector y , which is a $[16 \times 1]$ vector or a $[8 \times 1]$ vector, by a G matrix. For the inverse LFNST, the output vector x can be either a $[48 \times 1]$ vector or a $[16 \times 1]$ vector.

[0176] The output vector x is arranged in a two-dimensional block according to the order shown in FIG. 7 and is arranged as two-dimensional data, and this two-dimensional data becomes input data (or a part of input data) of the inverse primary transformation.

[0177] Accordingly, the inverse secondary transformation is the opposite of the forward secondary transformation process as a whole, and in the case of the inverse transformation, unlike in the forward direction, the inverse secondary transformation is first applied, and then the inverse primary transformation is applied.

[0178] In the inverse LFNST, one of 8 $[48 \times 16]$ matrices and 8 $[16 \times 16]$ matrices may be selected as the transformation matrix G . Whether to apply the $[48 \times 16]$ matrix or the $[16 \times 16]$ matrix depends on the size and shape of the block.

[0179] In addition, 8 matrices may be derived from four transform sets as shown in Table 2 above, and each transform set may consist of two matrices. Which transform set to use among the 4 transform sets is determined according to the intra prediction mode, and more specifically, the transform set is determined based on the value of the intra prediction mode extended by considering the Wide Angle Intra Prediction (WAIP). Which matrix to select from among the two matrices constituting the selected transform set is derived through index signaling. More specifically, 0, 1, and 2 are possible as the transmitted index value, 0 may indicate that the LFNST is not applied, and 1 and 2 may indicate any one of two transform matrices constituting a transform set selected based on the intra prediction mode value.

[0180] FIG. 9 is a diagram illustrating wide-angle intra prediction modes according to an embodiment of the present document.

[0181] The general intra prediction mode value may have values from 0 to 66 and 81 to 83, and the intra prediction mode value extended due to WAIP may have a value from -14 to 83 as shown. Values from 81 to 83 indicate the CCLM (Cross Component Linear Model) mode, and values from -14 to -1 and values from 67 to 80 indicate the intra prediction mode extended due to the WAIP application.

[0182] When the width of the prediction current block is greater than the height, the upper reference pixels are generally closer to positions inside the block to be predicted. Therefore, it may be more accurate to predict in the bottom-left direction than in the top-right direction. Conversely, when the height of the block is greater than the width, the left reference pixels are generally close to positions inside the block to be predicted. Therefore, it may be more accurate to predict in the top-right direction than in the bottom-left direction. Therefore, it may be advantageous to apply remapping, ie, mode index modification, to the index of the wide-angle intra prediction mode.

[0183] When the wide-angle intra prediction is applied, information on the existing intra prediction may be signaled, and after the information is parsed, the information may be remapped to the index of the wide-angle intra prediction

mode. Therefore, the total number of the intra prediction modes for a specific block (eg, a non-square block of a specific size) may not change, and that is, the total number of the intra prediction modes is 67, and intra prediction mode coding for the specific block may not be changed.

[0184] Table 3 below shows a process of deriving a modified intra mode by remapping the intra prediction mode to the wide-angle intra prediction mode.

TABLE 3

Inputs to this process are:

- a variable preModelIntra specifying the intra prediction mode,
- a variable nTbW specifying the transform block width,
- a variable nTbH specifying the transform block height,
- a variable cIdx specifying the colour component of the current block.

Output of this process is the modified intra prediction mode predModelIntra.

The variables nW and nH are derived as follows:

- If IntraSubPartitionsSplitType is equal to ISP_NO_SPLIT or cIdx is not equal to 0, the following applies:

$$nW = nTbW \quad (8-97)$$

$$nH = nTbH \quad (8-98)$$

- Otherwise (IntraSubPartitionsSplitType is not equal to ISP_NO_SPLIT and cIdx is equal to 0), the following applies:

$$nW = nCbW \quad (8-99)$$

$$nH = nCbH \quad (8-100)$$

The variable whRatio is set equal to $\text{Abs}(\text{Log2}(nW / nH))$.

For non-square blocks (nW is not equal to nH), the intra prediction mode predModelIntra is modified as follows:

- If all of the following conditions are true, predModelIntra is set equal to (predModelIntra + 65).
 - nW is greater than nH
 - predModelIntra is greater than or equal to 2
 - predModelIntra is less than (whRatio > 1) ? (8 + 2 * whRatio) : 8
- Otherwise, if all of the following conditions are true, predModelIntra is set equal to
 - (predModelIntra - 67).
 - nH is greater than nW
 - predModelIntra is less than or equal to 66
 - predModelIntra is greater than (whRatio > 1) ? (60 - 2 * whRatio) : 60

[0185] In Table 3, the extended intra prediction mode value is finally stored in the predModelIntra variable, and ISP_NO_SPLIT indicates that the CU block is not divided into sub-partitions by the Intra Sub Partitions (ISP) technique currently adopted in the VVC standard, and the cIdx variable Values of 0, 1, and 2 indicate the case of luma, Cb, and Cr components, respectively. Log2 function shown in Table 3 returns a log value with a base of 2, and the Abs function returns an absolute value.

[0186] Variable predModelIntra indicating the intra prediction mode and the height and width of the transform block, etc. are used as input values of the wide angle intra prediction mode mapping process, and the output value is the modified intra prediction mode predModelIntra. The height and width of the transform block or the coding block may be the height and width of the current block for remapping of the intra prediction mode. At this time, the variable whRatio reflecting the ratio of the width to the width may be set to $\text{Abs}(\text{Log2}(nW/nH))$.

[0187] For a non-square block, the intra prediction mode may be divided into two cases and modified.

[0188] First, if all conditions (1)~(3) are satisfied, (1) the width of the current block is greater than the height, (2) the intra prediction mode before modifying is equal to or greater than 2, (3) the intra prediction mode is less than the value derived from $(8+2*whRatio)$ when the variable whRatio is greater than 1, and is less than 8 when the variable whRatio is less than or equal to 1 [predModelIntra is less than $(whRatio>1)?(8+2*whRatio):8$], the intra prediction mode

is set to a value 65 greater than the intra prediction mode [predModelIntra is set equal to $(\text{predModelIntra}+65)$].

[0189] If different from the above, that is, follow conditions (1)~(3) are satisfied, (1) the height of the current block is greater than the width, (2) the intra prediction mode before modifying is less than or equal to 66, (3) the intra prediction mode is greater than the value derived from $(60-2*whRatio)$ when the variable whRatio is greater than 1, and is greater

than 60 when the variable whRatio is less than or equal to 1 [predModelIntra is greater than $(whRatio>1)?(60-2*whRatio):60$], the intra prediction mode is set to a value 67 smaller than the intra prediction mode [predModelIntra is set equal to $(\text{predModelIntra}-67)$].

[0190] Table 2 above shows how a transform set is selected based on the intra prediction mode value extended by the WAIP in the LFNST. As shown in FIG. 9, modes 14 to 33 and modes 35 to 80 are symmetric with respect to the prediction direction around mode 34. For example, mode 14 and mode 54 are symmetric with respect to the direction corresponding to mode 34.

[0191] Therefore, the same transform set is applied to modes located in mutually symmetrical directions, and this symmetry is also reflected in Table 2.

[0192] Meanwhile, it is assumed that forward LFNST input data for mode 54 is symmetrical with the forward LFNST input data for mode 14. For example, for mode 14 and mode 54, the two-dimensional data is rearranged into one-dimensional data according to the arrangement order shown in (a) of FIG. 7 and (b) of FIG. 7, respectively. In addition, it can be seen that the patterns in the order shown in (a) of FIG. 7 and (b) of FIG. 7 are symmetrical with respect to the direction (diagonal direction) indicated by Mode 34.

[0193] Meanwhile, as described above, which transform matrix of the $[48 \times 16]$ matrix and the $[16 \times 16]$ matrix is applied to the LFNST is determined by the size and shape of the transform target block.

[0194] FIG. 10 is a diagram illustrating a block shape to which the LFNST is applied. (a) of FIG. 10 shows 4×4 blocks, (b) shows 4×8 and 8×4 blocks, (c) shows 4×N or N×4 blocks in which N is 16 or more, (d) shows 8×8 blocks, (e) shows M×N blocks where M≥8, N>8, and N>8 or M>8.

[0195] In FIG. 10, blocks with thick borders indicate regions to which the LFNST is applied. For the blocks of FIGS. 10(a) and (b), the LFNST is applied to the top-left 4×4 region, and for the block of FIG. 10(c), the LFNST is applied individually the two top-left 4×4 regions are continuously arranged. In (a), (b), and (c) of FIG. 10, since the LFNST is applied in units of 4×4 regions, this LFNST will be hereinafter referred to as “4×4 LFNST”. As a corresponding transformation matrix, a [16×16] or [16×8] matrix may be applied based on the matrix dimension for G in Equations 9 and 10.

[0196] More specifically, the [16×8] matrix is applied to the 4×4 block (4×4 TU or 4×4 CU) of FIG. 10(a) and the [16×16] matrix is applied to the blocks in (b) and (c) of FIG. 10. This is to adjust the computational complexity for the worst case to 8 multiplications per sample.

[0197] With respect to (d) and (e) of FIG. 10, the LFNST is applied to the top-left 8×8 region, and this LFNST is hereinafter referred to as “8×8 LFNST”. As a corresponding transformation matrix, a [48×16] matrix or [48×8] matrix may be applied. In the case of the forward LFNST, since the [48×1] vector (x vector in Equation 9) is input as input data, all sample values of the top-left 8×8 region are not used as input values of the forward LFNST. That is, as can be seen in the left order of FIG. 7(a) or the left order of FIG. 7(b), the [48×1] vector may be constructed based on samples belonging to the remaining 3 4×4 blocks while leaving the bottom-right 4×4 block as it is.

[0198] The [48×8] matrix may be applied to an 8×8 block (8×8 TU or 8×8 CU) in FIG. 10 (d), and the [48×16] matrix may be applied to the 8×8 block in FIG. 10(e). This is also to adjust the computational complexity for the worst case to 8 multiplications per sample.

[0199] Depending on the block shape, when the corresponding forward LFNST (4×4 LFNST or 8×8 LFNST) is applied, 8 or 16 output data (y vector in Equation 9, [8×1] or [16×1] vector) is generated. In the forward LFNST, the number of output data is equal to or less than the number of input data due to the characteristics of the matrix GT.

[0200] FIG. 11 is a diagram illustrating an arrangement of output data of a forward LFNST according to an example, and shows a block in which output data of the forward LFNST is arranged according to a block shape.

[0201] The shaded area at the top-left of the block shown in FIG. 11 corresponds to the area where the output data of the forward LFNST is located, the positions marked with 0 indicate samples filled with 0 values, and the remaining area represents regions that are not changed by the forward LFNST. In the area not changed by the LFNST, the output data of the forward primary transform remains unchanged.

[0202] As described above, since the dimension of the transform matrix applied varies according to the shape of the block, the number of output data also varies. As FIG. 11, the output data of the forward LFNST may not completely fill the top-left 4×4 block. In the case of (a) and (d) of FIG. 11, a [16×8] matrix and a [48×8] matrix are applied to the block indicated by a thick line or a partial region inside the block, respectively, and a [8×1] vector as the output of the forward LFNST is generated. That is, according to the scan order

shown in (b) of FIG. 8, only 8 output data may be filled as shown in (a) and (d) of FIGS. 11, and 0 may be filled in the remaining 8 positions. In the case of the LFNST applied block of FIG. 10(d), as shown in FIG. 11(d), two 4×4 blocks in the top-right and bottom-left adjacent to the top-left 4×4 block are also filled with 0 values.

[0203] As described above, basically, by signaling the LFNST index, whether to apply the LFNST and the transform matrix to be applied are specified. As shown FIG. 11, when the LFNST is applied, since the number of output data of the forward LFNST may be equal to or less than the number of input data, a region filled with a zero value occurs as follows.

[0204] 1) As shown in (a) of FIG. 11, samples from the 8th and later positions in the scan order in the top-left 4×4 block, that is, samples from the 9th to the 16th.

[0205] 2) As shown in (d) and (e) of FIG. 11, when the [16×48] matrix or the [8×48] matrix is applied, two 4×4 blocks adjacent to the top-left 4×4 block or the second and third 4×4 blocks in the scan order.

[0206] Therefore, if non-zero data exists by checking the areas 1) and 2), it is certain that the LFNST is not applied, so that the signaling of the corresponding LFNST index can be omitted.

[0207] According to an example, for example, in the case of LFNST adopted in the VVC standard, since signaling of the LFNST index is performed after the residual coding, the encoding apparatus may know whether there is the non-zero data (significant coefficients) for all positions within the TU or CU block through the residual coding. Accordingly, the encoding apparatus may determine whether to perform signaling on the LFNST index based on the existence of the non-zero data, and the decoding apparatus may determine whether the LFNST index is parsed. When the non-zero data does not exist in the area designated in 1) and 2) above, signaling of the LFNST index is performed.

[0208] Since a truncated unary code is applied as a binarization method for the LFNST index, the LFNST index consists of up to two bins, and 0, 10, and 11 are assigned as binary codes for possible LFNST index values of 0, 1, and 2, respectively. In the case of the LFNST currently adopted for VVC, a context-based CABAC coding is applied to the first bin (regular coding), and a bypass coding is applied to the second bin. The total number of contexts for the first bin is 2, when (DCT-2, DCT-2) is applied as a primary transform pair for the horizontal and vertical directions, and a luma component and a chroma component are coded in a dual tree type, one context is allocated and another context applies for the remaining cases. The coding of the LFNST index is shown in a table as follows.

TABLE 4

Syntax element	binIdx					
	0	1	2	3	4	>=5
lfnst_idx[][]	(tu_mts_idx[x0][y0] == 0 && treeType != SINGLE_TREE) ? 1 : 0	by-pass	na	na	na	na

[0209] Meanwhile, for the adopted LFNST, the following simplification methods may be applied.

[0210] (i) According to an example, the number of output data for the forward LFNST may be limited to a maximum of 16.

[0211] In the case of (c) of FIG. 10, the 4×4 LFNST may be applied to two 4×4 regions adjacent to the top-left, respectively, and in this case, a maximum of 32 LFNST output data may be generated. When the number of output data for forward LFNST is limited to a maximum of 16, in the case of 4×N/N×4 (N>16) blocks (TU or CU), the 4×4 LFNST is only applied to one 4×4 region in the top-left, the LFNST may be applied only once to all blocks of FIG. 10. Through this, the implementation of image coding may be simplified.

[0212] FIG. 12 shows that the number of output data for the forward LFNST is limited to a maximum of 16 according to an example. As FIG. 12, when the LFNST is applied to the most top-left 4×4 region in a 4×N or N×4 block in which N is 16 or more, the output data of the forward LFNST becomes 16 pieces.

[0213] (ii) According to an example, zero-out may be additionally applied to a region to which the LFNST is not applied. In this document, the zero-out may mean filling values of all positions belonging to a specific region with a value of 0. That is, the zero-out can be applied to a region that is not changed due to the LFNST and maintains the result of the forward primary transformation. As described above, since the LFNST is divided into the 4×4 LFNST and the 8×8 LFNST, the zero-out can be divided into two types ((ii)-(A) and (ii)-(B)) as follows.

[0214] (ii)-(A) When the 4×4 LFNST is applied, a region to which the 4×4 LFNST is not applied may be zeroed out. FIG. 13 is a diagram illustrating the zero-out in a block to which the 4×4 LFNST is applied according to an example.

[0215] As shown in FIG. 13, with respect to a block to which the 4×4 LFNST is applied, that is, for all of the blocks in (a), (b) and (c) of FIG. 11, the whole region to which the LFNST is not applied may be filled with zeros.

[0216] On the other hand, (d) of FIG. 13 shows that when the maximum value of the number of the output data of the forward LFNST is limited to 16 as shown in FIG. 12, the zero-out is performed on the remaining blocks to which the 4×4 LFNST is not applied.

[0217] (ii)-(B) When the 8×8 LFNST is applied, a region to which the 8×8 LFNST is not applied may be zeroed out. FIG. 14 is a diagram illustrating the zero-out in a block to which the 8×8 LFNST is applied according to an example.

[0218] As shown in FIG. 14, with respect to a block to which the 8×8 LFNST is applied, that is, for all of the blocks in (d) and (e) of FIG. 11, the whole region to which the LFNST is not applied may be filled with zeros.

[0219] (iii) Due to the zero-out presented in (ii) above, the area filled with zeros may be not same when the LFNST is applied. Accordingly, it is possible to check whether the non-zero data exists according to the zero-out proposed in (ii) over a wider area than the case of the LFNST of FIG. 11.

[0220] For example, when (ii)-(B) is applied, after checking whether the non-zero data exists where the area filled with zero values in (d) and (e) of FIG. 11 in addition to the area filled with 0 additionally in FIG. 14, signaling for the LFNST index can be performed only when the non-zero data does not exist.

[0221] Of course, even if the zero-out proposed in (ii) is applied, it is possible to check whether the non-zero data exists in the same way as the existing LFNST index signal-

ing. That is, after checking whether the non-zero data exists in the block filled with zeros in FIG. 11, the LFNST index signaling may be applied. In this case, the encoding apparatus only performs the zero out and the decoding apparatus does not assume the zero out, that is, checking only whether the non-zero data exists only in the area explicitly marked as 0 in FIG. 11, may perform the LFNST index parsing.

[0222] Alternatively, according to another example, the zero-out may be performed as shown in FIG. 15. FIG. 15 is a diagram illustrating the zero-out in a block to which the 8×8 LFNST is applied according to another example.

[0223] As shown in FIGS. 13 and 14, the zero-out may be applied to all regions other than the region to which the LFNST is applied, or the zero-out may be applied only to a partial region as shown in FIG. 15. The zero-out is applied only to regions other than the top-left 8×8 region of FIG. 15, the zero-out may not be applied to the bottom-right 4×4 block within the top-left 8×8 region.

[0224] Various embodiments in which combinations of the simplification methods ((i), (ii)-(A), (ii)-(B), (iii)) for the LFNST are applied may be derived. Of course, the combinations of the above simplification methods are not limited to the following embodiments, and any combination may be applied to the LFNST.

EMBODIMENT

[0225] Limit the number of output data for forward LFNST to a maximum of 16> (i)

[0226] When the 4×4 LFNST is applied, all areas to which the 4×4 LFNST is not applied are zero-out→(ii)-(A)

[0227] When the 8×8 LFNST is applied, all areas to which the 8×8 LFNST is not applied are zero-out→(ii)-(B)

[0228] After checking whether the non-zero data exists also the existing area filled with zero value and the area filled with zeros due to additional zero outs ((ii)-(A), (ii)-(B)), the LFNST index is signaled only when the non-zero data does not exist→(iii)

[0229] In the case of Embodiment, when the LFNST is applied, an area in which the non-zero output data can exist is limited to the inside of the top-left 4×4 area. In more detail, in the case of FIG. 13(a) and FIG. 14(a), the 8th position in the scan order is the last position where non-zero data can exist. In the case of FIGS. 13(b) and (c) and FIG. 14(b), the 16th position in the scan order (ie, the position of the bottom-right edge of the top-left 4×4 block) is the last position where data other than 0 may exist.

[0230] Therefore, when the LFNST is applied, after checking whether the non-zero data exists in a position where the residual coding process is not allowed (at a position beyond the last position), it can be determined whether the LFNST index is signaled.

[0231] In the case of the zero-out method proposed in (ii), since the number of data finally generated when both the primary transform and the LFNST are applied, the amount of computation required to perform the entire transformation process can be reduced. That is, when the LFNST is applied, since zero-out is applied to the forward primary transform output data existing in a region to which the LFNST is not applied, there is no need to generate data for the region that become zero-out during performing the forward primary transform. Accordingly, it is possible to reduce the amount

of computation required to generate the corresponding data. The additional effects of the zero-out method proposed in (ii) are summarized as follows.

[0232] First, as described above, the amount of computation required to perform the entire transform process is reduced.

can be greatly reduced when LFNST is applied, the delay time for the entire transform is maintained or reduced, so that the encoding apparatus can be implemented more simply.

[0238] Hereinafter, the image decoding process in which the embodiment is reflected is shown in a table.

TABLE 5

7.3.2.3 Sequence parameter set RBSP syntax	
	Descriptor
seq_parameter_set_rbsp() {	
.....	
sps_log2_max_luma_transform_size_minus5	u(1)
sps_sao_enabled_flag	u(1)
sps_alf_enabled_flag	u(1)
sps_pcm_enabled_flag	u(1)
.....	
}	
7.4.3.3 Sequence parameter set RBSP semantics	
.....	
MinTbLog2SizeY = 2	(7-14)
MaxTbLog2SizeY = sps_log2_max_luma_transform_size_minus5 + 5	(7-15)
MinTbSizeY = 1 << MinTbLog2SizeY	(7-16)
MaxTbSizeY = 1 << MaxTbLog2SizeY	(7-17)
.....	
sps_log2_max_luma_transform_size_minus5 specifies the base 2 logarithm of the maximum transform size minus 5.	
.....	
sps_sbt_enabled_flag equal to 0 specifies that subblock transform for inter-predicted CUs is disabled.	
sps_sbt_enabled_flag equal to 1 specifies that subblock transform for inter-predicted CU is enabled.	
sps_sbt_max_size_64_flag equal to 0 specifies that the maximum CU width and height for allowing subblock transform is 32 luma samples. sps_sbt_max_size_64_flag equal to 1 specifies that the maximum CU width and height for allowing subblock transform is 64 luma samples.	
MaxSbtSize = Min(MaxTbSizeY, sps_sbt_max_size_64_flag ? 64 : 32)	(7-34)

[0233] In particular, when (ii)-(B) is applied, the amount of calculation for the worst case is reduced, so that the transform process can be lightened. In other words, in general, a large amount of computation is required to perform a large-size primary transformation. By applying (ii)-(B), the number of data derived as a result of performing the forward LFNST can be reduced to 16 or less. In addition, as the size of the entire block (TU or CU) increases, the effect of reducing the amount of transform operation is further increased.

[0234] Second, the amount of computation required for the entire transform process can be reduced, thereby reducing the power consumption required to perform the transform.

[0235] Third, the latency involved in the transform process is reduced.

[0236] The secondary transformation such as the LFNST adds a computational amount to the existing primary transformation, thus increasing the overall delay time involved in performing the transformation. In particular, in the case of intra prediction, since reconstructed data of neighboring blocks is used in the prediction process, during encoding, an increase in latency due to a secondary transformation leads to an increase in latency until reconstruction. This can lead to an increase in overall latency of intra prediction encoding.

[0237] However, if the zero-out suggested in (ii) is applied, the delay time of performing the primary transform

[0239] Table 5 shows that sps_log2_max_luma_transform_size_minus5, which is syntax information on the size of a transform block, is signaled through the sequence parameter set syntax. According to semantics, sps_log2_max_luma_transform_size_minus5 represents a value obtained by subtracting 5 after taking the base 2 logarithm for the maximum transform size.

[0240] The minimum size (MinTbSizeY) of a transform block in which transformation can be performed is set to 4 (MinTbSizeY=1<<MinTbLog2SizeY), and the maximum size of a transform block in which transformation can be performed may be derived as a power of 2 of a value obtained by adding 5 to sps_log2_max_luma_transform_size_minus5 (MaxTbLog2SizeY=sps_log2_max_luma_transform_size_minus5+5, MaxTbSizeY=1<<MaxTbLog2SizeY).

[0241] Since sps_log2_max_luma_transform_size_minus5 consists of 1 bit and has a value of 0 or 1, the width and height of the maximum transform block may be set to 32 or 64 based on sps_log2_max_luma_transform_size_minus5 of Table 5.

[0242] Meanwhile, according to another embodiment, flag information sps_max_luma_transform_size_64_flag for the size of the maximum transform block may be signaled. When sps_max_luma_transform_size_64_flag is 1, the maximum size of the transform block is 64, and when sps_max_luma_transform_size_64_flag is 0, the maximum size of the transform block is 32.

TABLE 6

7.3.7.5 Coding unit syntax	
	Descriptor
coding_unit(x0, y0, cbWidth, cbHeight, treeType) {	
.....	
if(!pcm_flag[x0][y0]) {	
if(CuPredMode[x0][y0] != MODE_INTRA &&	
general_merge_flag[x0][y0] == 0)	
cu_cbf	ae(v)
if(cu_cbf) {	
.....	
LfstDcLastScanPos = 1	
LfstZeroOutSigCoeffFlag = 1	
transform_tree(x0, y0, cbWidth, cbHeight, treeType)	
lfstWidth = (treeType == DUAL_TREE_CHROMA) ? cbWidth / Sub-	
WidthC	
: cbWidth	
lfstHeight = (treeType == DUAL_TREE_CHROMA) ? cbHeight /	
SubHeightC	
: cbHeight	
if(Min(lfstWidth, lfstHeight) >= 4 && sps_lfst_enabled_flag == 1 &&	
CuPredMode[x0][y0] == MODE_INTRA &&	
IntraSubPartitionsSplitType == ISP_NO_SPLIT &&	
!intra_mip_flag[x0][y0] && tu_mts_idx[x0][y0] == 0 &&	
Max(cbWidth, cbHeight) <= MaxTbSizeY) {	
if(LfstDcOnly == 0 && LfstZeroOutSigCoeffFlag == 1)	
lfst_idx[x0][y0]	ae(v)
}	
}	
}	

[0243] Table 6 shows lfst_idx [x0] [y0] syntax elements signaled at the coding unit level. lfst_idx [x0] [y0] may indicate any one of two transform kernel matrices included in the transform set. If lfst_idx is 0, it may indicate that non-separable secondary transform, that is, LENST is not applied.

[0244] In order for lfst_idx to be parsed by the decoding apparatus, many conditions must be satisfied. First, the variable LfstDcOnly and the variable LfstZeroOutSigCoeffFlag are initially set to 1. After parsing the syntax for the transformation tree (transform_tree (x0, y0, cbWidth, cbHeight, treeType)), when the variable LfstDcOnly set to 1 is changed to 0, and the value of the variable LfstZeroOutSigCoeffFlag is maintained at 1, lfst_idx can be parsed [if (LfstDcOnly==0 && LfstZeroOutSigCoeffFlag==1)]. The variable LfstDcOnly and the variable LfstZeroOutSigCoeffFlag may be derived through residual coding syntax information.

[0245] Meanwhile, the maximum coding block size in which lfst_idx [x0] [y0] can be coded is limited to the maximum transform size (Max (cbWidth, cbHeight) <=MaxTbSize Y).

[0246] In addition, since the width (cbWidth) of the coding block and the height (cbHeight) of the coding block indicate the width of the coding block and the height of the coding block for the luma component, respectively, in the case of the chroma component, the LFNST may be applied to each block having a smaller size according to the image color format (eg, 4:2:0).

[0247] Specifically, as shown in Table 6, if the tree type of the target block is dual tree chroma, the LFNST may be applied to a chroma block having a size divided by SubWidthC and SubHeight indicating a variable for the chroma format in the size of the luma coding block [lfstWidth=

(treeType==DUAL_TREE_CHROMA)? cbWidth/SubWidthC: cb Width, lfstHeight (treeType DUAL_TREE_CHROMA)? cbHeight/SubHeightC: cbHeight].

[0248] If the color format is 4:2:0, Sub WidthC and SubHeight become 2, so the LFNST may be applied to a chroma block having a width and height obtained by dividing the width and height of the luma block by 2. Therefore, since the LFNST can be applied when the size of the luma block is equal to or smaller than the 64×64 block, the LFNST can be applied when the size of the chroma block is equal to or smaller than the 32×32 block when the color format is 4:2:0.

[0249] Meanwhile, in this document, when the horizontal and vertical lengths of block A are Wa and Ha, respectively, and when the horizontal and vertical lengths of block B are Wb and Hb, respectively, block A is smaller than block B means that Wa is equal to or smaller than Wb, Ha is equal to or smaller than Hb, and Wa and Wb are not equal or Ha and Hb are not equal. Also, the meaning that block A is smaller than or equal to block B indicates that Wa is equal to or smaller than Wb and Ha is equal to or smaller than Hb.

[0250] In summary, when the size of the target block is equal to or smaller than the preset maximum size, the LFNST may be applied, this maximum size may be applied to the size of the luma block, and correspondingly, the maximum size of the chroma block to which the LFNST may be applied may be derived.

TABLE 7

7.3.7.10 Transform unit syntax	Descriptor
<pre> transform_unit(x0, y0, tbWidth, tbHeight, treeType, subTuIndex) { if(tu_cbf_luma[x0][y0] && treeType != DUAL_TREE_CHROMA && (tbWidth <= 32) && (tbHeight <= 32) && (IntraSubPartitionsSplit[x0][y0] == ISP_NO_SPLIT) && (!cu_sbt_flag)) { if(transform_skip_enabled_flag && tbWidth <= MaxTsSize && tbHeight <= MaxTsSize) transform_skip_flag[x0][y0] if(((CuPredMode[x0][y0] == MODE_INTER && sps_explicit_mts_inter_enabled_ flag) (CuPredMode[x0][y0] == MODE_INTRA && sps_explicit_mts_intra_ enabled_flag) && (!transform_skip_flag[x0][y0]))) tu_mts_idx[x0][y0] } } } } </pre>	ae(v) ae(v)

[0251] Table 7 shows transform_skip_flag indicating whether to skip transform with respect to a transform block and tu_mts_idx [x0] [y0], which is transform kernel index information for primary transform.

[0252] As shown in Table 7, in order for tu_mts_idx [x0] [y0] to be signaled, there may be a case where the prediction mode of the current block is inter mode or or when flag information sps_explicit_mts_inter_enabled_flag that explicitly indicates whether the MTS can be applied to residual data generated by the inter prediction is 1 [(CuPred-Mode [x0] [y0]==MODE_INTER && sps_explicit_mts_inter_enabled_flag)], or a case where the prediction mode of

the current block is intra mode or or when flag information `sps_explicit_mts_intra_enabled_flag` that explicitly indicates whether the MTS can be applied to residual data generated by the intra prediction is 1 [(CuPredMode[x0][y0]==MODE_INTRA && sps_explicit_mts_intra_enabled_flag)].

[0253] Additionally, when a condition in which transform_skip_flag is not 0 is satisfied, tu_mts_idx [x0] [y0] may be parsed.

[0254] Meanwhile, according to another example, the tu_mts_idx [x0] [y0] may be signaled at the coding unit level of Table 6 rather than the transform unit level.

TABLE 8

7.3.7.11 Residual coding syntax
Descriptor

```

residual_coding( x0, y0, log2TbWidth, log2TbHeight, cIdx ) {
    if( ( tu_mts_idx[ x0 ][ y0 ] > 0 ) ||
        ( cu_sbt_flag && log2TbWidth < 6 && log2TbHeight < 6 ) )
        && cIdx = 0 && log2TbWidth > 4 )
        log2ZoTbWidth = 4
    else
        log2ZoTbWidth = Min( log2TbWidth, 5 )
    MaxCobs = 2 * ( 1 << log2TbWidth ) * ( 1 << log2TbHeight )
    if( tu_mts_idx[ x0 ][ y0 ] > 0 ) ||
        ( cu_sbt_flag && log2TbWidth < 6 && log2TbHeight < 6 ) )
        && cIdx = 0 && log2TbHeight > 4 )
        log2ZoTbHeight = 4
    else
        log2ZoTbHeight = Min( log2TbHeight, 5 )
    if( log2TbWidth > 0 )

```

TABLE 8-continued

7.3.7.11 Residual coding syntax	
	Descriptor
last_sig_coeff_x_prefix	ae(v)
if(log2TbHeight > 0)	
last_sig_coeff_y_prefix	ae(v)
if(last_sig_coeff_x_prefix > 3)	
last_sig_coeff_x_suffix	ae(v)
if(last_sig_coeff_y_prefix > 3)	
last_sig_coeff_y_suffix	ae(v)
log2TbWidth = log2ZoTbWidth	
log2TbHeight = log2ZoTbHeight	
log2SbW = (Min(log2TbWidth, log2TbHeight) < 2 ? 1 : 2)	
log2SbH = log2SbW	
if(log2TbWidth + log2TbHeight > 3) {	
if(log2TbWidth < 2) {	
log2SbW = log2TbWidth	
log2SbH = 4 - log2SbW	
} else if(log2TbHeight < 2) {	
log2SbH = log2TbHeight	
log2SbW = 4 - log2SbH	
}	
}	
numSbCoeff = 1 << (log2SbW + log2SbH)	
lastScanPos = numSbCoeff	
lastSubBlock = (1 << (log2TbWidth + log2TbHeight - (log2SbW + log2SbH))) - 1	
do {	
if lastScanPos == 0) {	
lastScanPos = numSbCoeff	
lastSubBlock--	
}	
lastScanPos--	
xS = DiagScanOrder[log2TbWidth - log2SbW][log2TbHeight - log2SbH]	
[last SubBlock][0]	
yS = DiagScanOrder[log2TbWidth - log2SbW][log2TbHeight - log2SbH]	
[last SubBlock][1]	
xC = (xS << log2SbW) + DiagScanOrder[log2SbW][log2SbH]	
[lastScanPos][0]	
yC = (yS << log2SbH) + DiagScanOrder[log2SbW][log2SbH]	
[lastScanPos][1]	
} while((xC != LastSignificantCoeffX) (yC != LastSignificantCoeffY))	
if(lastSubBlock > 0 && log2TbWidth >= 2 && log2TbHeight >= 2 &&	
!transform_skip_flag[x0][y0] && lastScanPos > 0) {	
LfnstDcOnly = 0	
}	
if((lastSubBlock > 0 && last2TbWidth >= 2 && log2TbHeight >= 2)	
(lastScanPos > 7 && (log2TbWidth == 2 log2TbHeight == 3) &&	
log2TbWidth == log2TbHeight)) {	
LfnstZeroOutSigCoeffFlag = 0	
}	
.....	

[0255] Table 8 shows a residual coding syntax, and the process in which the variable LfnstDcOnly and the variable LfnstZeroOutSigCoeffFlag of Table 6 are derived is shown.

[0256] Based on the size of the transform block, a variable log 2SbW and a variable log 2SbH indicating the height and width of the sub-block can be derived, numSbCoeff representing the number of coefficients that may exist in a sub-block may be set based on the variable log 2SbW and the variable log 2SbH [numSbCoeff=1<<(log 2SbW+log 2SbH)].

[0257] A variable lastScanPos indicating the position of the last significant coefficient within the subblock is initially set to numSbCoeff, a variable lastSubBlock indicating the subblock in which the last non-zero coefficient exists is initially set to “(1<<(log 2TbWidth+log 2TbHeight-(log 2SbW+log 2SbH)))-1”.

[0258] While scanning diagonally in the subblock corresponding to lastSubBlock [lastScanPos--], it is checked whether the last non-zero significant coefficient exists at the corresponding position.

[0259] When no significant coefficient is found until the variable lastScanPos becomes 0 within the subblock pointed to by lastSubBlock, the variable lastScanPos is again set to numSbCoeff, and the variable lastSubBlock is also changed to the next sub-block in the scan direction.

[0260] That is, as the variable lastScanPos and the variable lastSubBlock are updated according to the scan direction, the position where the last non-zero coefficient exists is identified.

[0261] The variable LfnstDcOnly indicates whether a non-zero coefficient exists at a position that is not a DC component for at least one transform block in one coding unit, when the non-zero coefficient exists in the position that is

not the DC component for at least one transform block in the one coding unit, it becomes 0, and when non-zero coefficients do not exist at positions other than the DC components for all transform blocks in the one coding unit it becomes 1. In this document, the DC component refers to (0, 0) or the top-left position relative to the 2D component.

[0262] Several transform blocks may exist within one coding unit. For example, in the case of a chroma component, transform blocks for Cb and Cr may exist, and in the case of a single tree type, transform blocks for luma, Cb, and Cr may exist. According to an example, when a non-zero coefficient other than the position of the DC component is found in one transform block among transform blocks constituting the current coding block, the variable LfnstDcOnly value may be set to 0.

[0263] Meanwhile, since the residual coding is not performed on the corresponding transform block if the non-zero coefficient do not exist in the transform block, the value of the variable LfnstDcOnly is not changed by the corresponding transform block. Therefore, when the non-zero coefficient does not exist in the position that is not the DC component in the transform block, the value of the variable LfnstDcOnly is not changed and the previous value is maintained. For example, if the coding unit is coded as a single tree type and the variable LfnstDcOnly value is changed to 0 due to the luma transform block, when the non-zero coefficient exist only in the DC component in the Cb transform block or the non-zero coefficient do not exist in the Cb transform block, the value of the variable LfnstDcOnly remains 0. The value of the variable LfnstDcOnly is initially initialized to 1, and if no component in the current coding unit can update the value of the variable LfnstDcOnly to 0, it maintains the value of 1 as it is and when the value of the variable LfnstDcOnly is updated to 0 at any one of the transform blocks constituting the corresponding coding unit, it is finally maintained at 0.

[0264] As shown in Table 8, when the index of the subblock in which the last non-zero coefficient exists is 0 [lastSubBlock=0] and the position of the last non-zero coefficient in the sub-block is greater than 0 [lastScanPos>0], the variable LfnstDcOnly may be derived as 0. The variable LfnstDcOnly can be derived as 0 only when the width and height of the transform block are 4 or more [$\log 2TbWidth \geq 2$ && $\log 2TbHeight \geq 2$], and no transform skip is applied [!transform_skip_flag[x0][y0]].

[0265] Assuming that the LFNST is applied, the variable LfnstZeroOutSigCoeffFlag, which can indicate whether the zero-out was performed properly, is set to 0 in case that the index of the sub-block in which the last non-zero coefficient exists is greater than 0, and the width and height of the transform block are both greater than or equal to 4 [$(lastSubBlock > 0 \ \&\& \ \log 2TbWidth \geq 2 \ \&\& \ \log 2TbHeight \geq 2)$], or in case that when the last position of the non-zero coefficient within the subblock in which the last non-zero coefficient exists is greater than 7 and the size of the transform block is 4x4 or 8x8 [$(lastScanPos > 7 \ \&\& \ (\log 2TbWidth = 2 \parallel \log 2TbHeight = 3) \ \&\& \ \log 2TbWidth = \log 2TbHeight)$].

[0266] That is, the first condition for the variable LfnstZeroOutSigCoeffFlag is a condition in which the non-zero coefficient is derived in the region other than the top-left region to which the LFNST can be applied in the transform block (That is, when the significant coefficient in sub-blocks other than the top-left sub-block (4x4) are derived). When the first condition is satisfied, the flag variable LfnstZeroOutSigCoeffFlag for the zero out of the LFNST is set to 0. Satisfying the first condition indicates that zero-out is not performed, assuming that that LFNST is applied.

[0267] The second condition for the variable LfnstZeroOutSigCoeffFlag is for the 4x4 block and the 8x8 block. When the LFNST is applied to the 4x4 block and the 8x8 block, since the last position where the non-zero coefficient can exist is the 8th position as shown in (a) and (d) of FIG. 11, if the non-zero coefficient exists outside the 7th position when starting from 0, the flag variable LfnstZeroOutSigCoeffFlag is set to 0. Satisfying the second condition also indicates that zero-out is not performed when it is assumed that the LFNST is applied.

[0268] As such, when the flag variable LfnstZeroOutSigCoeffFlag is set to 0, as shown in Table 6, lfnst_idx signaled at the coding unit level is not signaled. That is, when the flag variable LfnstZeroOutSigCoeffFlag is set to 0, the decoding apparatus does not parse lfnst_idx.

[0269] In summary, at the coding unit level, the variable LfnstDcOnly and the variable LfnstZeroOutSigCoeffFlag are set to 1, respectively, and then are newly derived through the process shown in Table 8 at the residual coding level. Only when the variable LfnstDcOnly derived from the residual coding level is 0 and the variable LfnstZeroOutSigCoeffFlag is 1 [if (LfnstDcOnly=0 && LfnstZeroOutSigCoeffFlag=1), lfnst_idx may be signaled.

TABLE 9

8.4.5 Decoding process for intra blocks
8.4.5.1 General decoding process for intra blocks

Inputs to this process are:

- a sample location (xTb0, yTb0) specifying the top-left sample of the current transform block relative to the top-left sample of the current picture,
- a variable nTbW specifying the width of the current transform block,
- a variable nTbH specifying the height of the current transform block,
- a variable predModeIntra specifying the intra prediction mode,
- a variable cIdx specifying the colour component of the current block.

Output of this process is a modified reconstructed picture before in-loop filtering.

The maximum transform block size maxTbSize is derived as follows:

$$\begin{aligned} \text{maxTbWidth} &= (\text{cIdx} == 0) ? \text{MaxTbSizeY} : \text{MaxTbSizeY} / \text{SubWidthC} \\ \text{maxTbHeight} &= (\text{cIdx} == 0) ? \text{MaxTbSizeY} : \text{MaxTbSizeY} / \text{SubHeightC} \end{aligned}$$

The luma sample location is derived as follows:

$$\begin{aligned} (\text{xTbY}, \text{yTbY}) &= (\text{cIdx} == 0) ? (\text{xTb0}, \text{yTb0}) : (\text{xTb0} * \text{SubWidthC}, \text{yTb0} * \\ &\quad \text{SubHeightC}) \end{aligned} \quad (8-50)$$

TABLE 9-continued

8.4.5 Decoding process for intra blocks	
8.4.5.1 General decoding process for intra blocks	

Depending on maxTbSize, the following applies:

- If IntraSubPartitionsSplitType is equal to NO_ISP_SPLIT and nTbW is greater than maxTbWidth or nTbH is greater than maxTbHeight, the following ordered steps apply.
 1. The variables newTbW and newTbH are derived as follows:

$$\text{newTbW} = (\text{nTbW} > \text{maxTbWidth}) ? (\text{nTbW} / 2) : \text{nTbW} \quad (8-51)$$

$$\text{newTbH} = (\text{nTbH} > \text{maxTbHeight}) ? (\text{nTbH} / 2) : \text{nTbH} \quad (8-52)$$
 2. The general decoding process for intra blocks as specified in this clause is invoked with the location (xTb0, yTb0), the transform block width nTbW set equal to newTbW and the height nTbH set equal to newTbH, the intra prediction mode predModeIntra and the variable cIdx as inputs, and the output is a modified reconstructed picture before in-loop filtering.
 3. If nTbW is greater than maxTbWidth, the general decoding process for intra blocks as specified in this clause is invoked with the location (xTb0, yTb0) set equal to (xTb0 + newTbW, yTb0), the transform block width nTbW set equal to newTbW and the height nTbH set equal to newTbH, the intra prediction mode predModeIntra, and the variable cIdx as inputs, and the output is a modified reconstructed picture before in-loop filtering.
 4. If nTbH is greater than maxTbHeight, the general decoding process for intra blocks as specified in this clause is invoked with the location (xTb0, yTb0) set equal to (xTb0, yTb0 + newTbH), the transform block width nTbW set equal to newTbW and the height nTbH set equal to newTbH, the intra prediction mode predModeIntra, and the variable cIdx as inputs, and the output is a modified reconstructed picture before in-loop filtering.
 5. If nTbW is greater than maxTbWidth and nTbH is greater than maxTbHeight, the general decoding process for intra blocks as specified in this clause is invoked with the location (xTb0, yTb0) set equal to (xTb0 + newTbW, yTb0 + newTbH), the transform block width nTbW set equal to newTbW and the height nTbH set equal to newTbH, the intra prediction mode predModeIntra, and the variable cIdx as inputs, and the output is a modified reconstructed picture before in-loop filtering.
- Otherwise, the following ordered steps apply:

.....

TABLE 10

8.5.8 Decoding process for the residual signal of coding blocks coded in inter prediction mode	
--	--

Inputs to this process are:

- a sample location (xTb0, yTb0) specifying the top-left sample of the current transform block relative to the top-left sample of the current picture,
- a variable nTbW specifying the width of the current transform block,
- a variable nTbH specifying the height of the current transform block,
- a variable cIdx specifying the colour component of the current block.

Output of this process is an (nTbW)x(nTbH) array resSamples.

The maximum transform block size maxTbSize is derived as follows:

$$\text{maxTbWidth} = (\text{cIdx} == 0) ? \text{MaxTbSizeY} : \text{MaxTbSizeY} / \text{SubWidthC}$$

$$\text{maxTbHeight} = (\text{cIdx} == 0) ? \text{MaxTbSizeY} : \text{MaxTbSizeY} / \text{SubHeightC}$$

The luma sample location is derived as follows:

$$(\text{xTbY}, \text{yTbY}) = (\text{cIdx} == 0) ? (\text{xTb0}, \text{yTb0}) : (\text{xTb0} * \text{SubWidthC}, \text{yTb0} * \text{SubHeightC}) \quad (8-868)$$

Depending on maxTbSize, the following applies:

- If nTbW is greater than maxTbWidth or nTbH is greater than maxTbHeight, the following ordered steps apply.
 1. The variables newTbW and newTbH are derived as follows:

$$\text{newTbW} = (\text{nTbW} > \text{maxTbWidth}) ? (\text{nTbW} / 2) : \text{nTbW} \quad (8-869)$$

$$\text{newTbH} = (\text{nTbH} > \text{maxTbHeight}) ? (\text{nTbH} / 2) : \text{nTbH} \quad (8-870)$$
 2. The decoding process for the residual signal of coding units coded in inter prediction mode as specified in this clause is invoked with the location (xTb0, yTb0), the transform block width nTbW set equal to newTbW and the height nTbH set equal to newTbH, and the variable cIdx as inputs, and the output is a modified reconstructed picture before in-loop filtering.
 3. When nTbW is greater than maxTbWidth, the decoding process for the residual signal of coding units coded in inter prediction mode as specified in this clause is invoked with the location (xTb0, yTb0) set equal to (xTb0 + newTbW, yTb0), the transform block width nTbW set equal to newTbW and the height nTbH set equal to newTbH, and the variable cIdx as inputs, and the output is a modified reconstructed picture .
 4. When nTbH is greater than maxTbHeight, the decoding process for the residual signal of coding units coded in inter prediction mode as specified in this clause is invoked with the location (xTb0, yTb0) set equal to (xTb0, yTb0 + newTbH), the transform block width nTbW set equal to newTbW and the height nTbH set equal to newTbH, and the variable cIdx as inputs, and the output is a modified reconstructed picture before in-loop filtering.

TABLE 10-continued

8.5.8 Decoding process for the residual signal of coding blocks coded in inter prediction mode
5. When nTbW is greater than maxTbWidth and nTbH is greater than maxTbHeight, the decoding process for the residual signal of coding units coded in inter prediction mode as specified in this clause is invoked with the location (xTb0, yTb0) set equal to (xTb0 + newTbW, yTb0 + newTbH), the transform block width nTbW set equal to newTbW and height nTbH set equal to newTbH, and the variable cIdx as inputs, and the output is a modified reconstructed picture before in-loop filtering. Otherwise, the scaling and transformation process as specified in clause 8.7.2 is invoked with the luma location (xTbY, yTbY), the variable cIdx, the transform width nTbW and the transform height nTbH as inputs, and the output is an (nTbW)x(nTbH) array resSamples.

[0270] Tables 9 and 10 show that intra prediction and inter prediction processes are performed based on the variable MaxTbSize Y for the size of the transform block derived from Table 5.

[0271] The maximum width (max TbWidth) and maximum height (max TbHeight) of the transform block are derived from either the variable MaxTbSize Y or MaxTbSize Y/Sub WidthC that reflects the color format according to the color index cIdx for luma or chroma [maxTbWidth=

(cIdx==0)? MaxTbSizeY: MaxTbSize Y/Sub WidthC, max TbHeight=(cIdx==0)? MaxTbSizeY: MaxTbSize Y/Sub HeightC].

[0272] The height and width of the transform block for the inter prediction and the intra prediction are set based on the variable maxTbWidth and the variable maxTbHeight derived in this way [newTbW=(nTbW>max TbWidth)? (nTbW/2): nTbW, newTbH=(nTbH>maxTbHeight)? (nTbH/2): nTbH], a subsequent prediction process may be performed based on the set value.

TABLE 11

8.7.4 Transformation process for scaled transform coefficients
8.7.4.1 General
Inputs to this process are: - a luma location (xTbY, yTbY) specifying the top-left sample of the current luma transform block relative to the top-left luma sample of the current picture, - a variable nTbW specifying the width of the current transform block, - a variable nTbH specifying the height of the current transform block, - a variable cIdx specifying the colour component of the current block, - an (nTbW)x(nTbH) array d[x][y] of scaled transform coefficients with x = 0..nTbW - 1, y = 0..nTbH - 1. Output of this process is the (nTbW)x(nTbH) array r[x][y] of residual samples with x = 0..nTbW - 1, y = 0..nTbH - 1. When lfst_idx[xTbY][yTbY] is not equal to 0 and both nTbW and nTbH are greater than or equal to 4, the following applies: - The variables predModeIntra, nLfstOutSize, log2LfstSize, nLfstSize, and nonZeroSize are derived as follows: <pre> predModeIntra = (cIdx == 0) ? IntraPredModeY[xTbY][yTbY] : IntraPredModeC[xTbY][yTbY] (8-952) nLfstOutSize = (nTbW >= 8 && nTbH >= 8) ? 48 : 16 (8-953) log2LfstSize = (nTbW >= 8 && nTbH >= 8) ? 3 : 2 (8-954) nLfstSize = 1 << log2LfstSize (8-955) nonZeroSize = ((nTbW == 4 && nTbH == 4) (nTbW == 8 && nTbH == 8)) ? 8 : 16 (8-958) </pre> - When predModeIntra is greater than or equal to 81, the chroma intra prediction mode derivation process as specified in clause 8.4.4 is invoked with a luma location (xTbY, yTbY), nTbW, nTbH as inputs, and predModeIntra is set equal to IntraPredModeY[xTbY + nTbW / 2][yTbY + nTbH / 2]. - The wide angle intra prediction mode mapping process as specified in clause 8.4.5.2.6 is invoked with predModeIntra, nTbW, nTbH and cIdx as inputs, and the modified predModeIntra as output. - The values of the list u[x] with x = 0..nonZeroSize - 1 are derived as follows: <pre> xC = DiagScanOrder[2][2][x][0] (8-959) yC = DiagScanOrder[2][2][x][1] (8-960) u[x] = d[xC][yC] (8-961) </pre> - The one-dimensional low frequency non-separable transformation process as specified in clause 8.7.4.2 is invoked with the input length of the scaled transform coefficients nonZeroSize, the transform output length nTrS set equal to nLfstOutSize the list of scaled non-zero transform coefficients u[x] with x = 0..nonZeroSize - 1, the intra prediction mode for LFNST set selection preModeIntra, and the LFNST index for transform selection in the selected LFNST set lfst_idx[xTbY][yTbY] as inputs, and the list v[x] with x = 0..nLfstOutSize - 1 as output.

TABLE 11-continued

8.7.4 Transformation process for scaled transform coefficients	
8.7.4.1 General	
- The array $d[x][y]$ with $x = 0..nLfstSize - 1$, $y = 0..nLfstSize - 1$ is derived as follows: - If $predModeIntra$ is less than or equal to 34, the following applies: $d[x][y] = (y < 4) ? v[x + (y << \log2LfstSize)] : ((x < 4) ? v[32 + x + ((y - 4) << 2)] : d[x][y]) \quad (8-962)$ - Otherwise, the following applies: $d[x][y] = (x < 4) ? v[y + (x << \log2LfstSize)] : ((y < 4) ? v[32 + y + ((x - 4) << 2)] : d[x][y]) \quad (8-963)$ The variable $implicitMtsEnabled$ is derived as follows: - If $sps_mts_enabled_flag$ is equal to 1 and one of the following conditions is true, $implicitMtsEnabled$ is set equal to 1: - $IntraSubPartitionsSplitType$ is not equal to ISP_NO_SPLIT - cu_sbt_flag is equal to 1 and $Max(nTbW, nTbH)$ is less than or equal to 32 - $sps_explicit_mts_intra_enabled_flag$ is equal to 0 and $CuPredMode[xTbY][yTbY]$ is equal to $MODE_INTRA$ and $lfst_idx[x0][y0]$ is equal to 0 and $intra_mip_flag[x0][y0]$ is equal to 0 - Otherwise, $implicitMtsEnabled$ is set equal to 0. The variable $trTypeHor$ specifying the horizontal transform kernel and the variable $trTypeVer$ specifying the vertical transform kernel are derived as follows: - If $cldx$ is greater than 0, $trTypeHor$ and $trTypeVer$ are set equal to 0. - Otherwise, if $implicitMtsEnabled$ is equal to 1, the following applies: - If $IntraSubPartitionsSplitType$ is not equal to ISP_NO_SPLIT or both $sps_explicit_mts_intra_enabled_flag$ and $sps_explicit_mts_inter_enabled_flag$ are equal to 0 and $CuPredMode[xTbY][yTbY]$ is equal to $MODE_INTRA$, $trTypeHor$ and $trTypeVer$ are derived as follows: $trTypeHor = (nTbW \geq 4 \ \&\& \ nTbW \leq 16) ? 1 : 0 \quad (8-964)$ $trTypeVer = (nTbH \geq 4 \ \&\& \ nTbH \leq 16) ? 1 : 0 \quad (8-965)$ - Otherwise, if cu_sbt_flag is equal to 1, $trTypeHor$ and $trTypeVer$ are specified in Table 8-19 depending on $cu_sbt_horizontal_flag$ and $cu_sbt_pos_flag$. - Otherwise, $trTypeHor$ and $trTypeVer$ are specified in Table 8-18 depending on $tu_mts_idx[xTbY][yTbY]$. The variables $nonZeroW$ and $nonZeroH$ are derived as follows: - If $lfst_idx[xTbY][yTbY]$ is not equal to 0, the following applies: $nonZeroW = (nTbW == 4 \ \ nTbH == 4) ? 4 : 8$ $nonZeroH = (nTbW == 4 \ \ nTbH == 4) ? 4 : 8$ - Otherwise, the following applies: $nonZeroW = \min(nTbW, (trTypeHor > 0) ? 16 : 32) \quad (8-966)$ $nonZeroH = \min(nTbH, (trTypeVer > 0) ? 16 : 32) \quad (8-967)$	

[0273] Table 11 shows the overall conversion process performed by the decoding apparatus.

[0274] Referring to Table 11, the variable $nonZeroSize$ indicating the size or number of non-zero variables on which matrix operation is performed in order to apply LFNST is set to 8 or 16. When the width and height of the transform block are 4 or 8, that is, the length of the output data of the forward LFNST or the input data of the inverse LFNST of the 4x4 block and the 8x8 block as shown in FIG. 11 is 8. For all other blocks, the length of the output data of the forward LFNST or the input data of the inverse LFNST is 16 [$nonZeroSize = ((nTbW == 4 \ \&\& \ nTbH == 4) \ || \ (nTbW == 8 \ \&\& \ nTbH == 8)) ? 8 : 16$]. That is, when the forward LFNST is applied, the maximum number of output data is limited to 16. The input data of this inverse LFNST may be two-dimensionally arranged according to a diagonal scan [$xC = \text{DiagScanOrder}[2][2][x][0]$, $yC = \text{DiagScanOrder}[2][2][x][1]$]. The above-described part shows the decoding process for (i) of the LFNST simplification method.

[0275] As such, since the number of input data of the inverse LFNST for the transform block is limited to a maximum of 16, the LFNST can be applied to the most top-left 4x4 region in a 4xN block or a Nx4 block where N is 16 or more, as shown in FIG. 12 and as a result, as shown in (d) of FIG. 13, the zero-out may be performed on the remaining blocks to which the 4x4 LFNST is not applied.

[0276] On the other hand, when the intra prediction mode is greater than or equal to 81, that is, when CCLM is applied

during the intra prediction of the chroma block, an intra prediction mode ($predModeIntra$) for deriving a transform set may be set as an intra mode ($IntraPredModeY[xTbY + nTbW/2][yTbY + nTbH/2]$) of a corresponding luma block. [0277] Meanwhile, a variable $implicitMtsEnabled$ indicating whether the MTS is implicitly performed may be set to 1 when it satisfies the conditions that flag information $sps_mts_enabled_flag$ signaled at the sequence parameter level is 1, $sps_explicit_mts_intra_enabled_flag$ is 0, the intra prediction mode is applied to the current block, $lfst_idx$ is 0, and $intra_mip_flag$ is 1.

[0278] In addition, variables $nonZeroW$ and $nonZeroH$ indicating the width and height of the upper-left block in which the non-zero transform coefficient input to the inverse primary transform can exist are derived as 4 when the $lfst$ index is not 0 and the width or height of the transform block is 4, otherwise is derived as 8 [$nonZeroW = (nTbW == 4 \ || \ nTbH == 4) ? 4 : 8$, $nonZeroH = (nTbW == 4 \ || \ nTbH == 4) ? 4 : 8$]. That is, in the transform block, the zero-out is performed in the regions other than the 4x4 region and the 8x8 region to which the $lfst$ is applied. This part shows the decoding process for (ii) of the LFNST simplification method.

TABLE 12

8.7.4.3 Low frequency non-separable transformation matrix derivation process	
Inputs to this process are:	
- a variable nTrS specifying the transform output length,	
- a variable predModeIntra specifying the intra prediction mode for LFNST set selection,	
- a variable lfnstIdx specifying the LFNST index for transform selection in the selected LFNST set.	
Output of this process is the transformation matrix lowFreqTransMatrix.	
The variable lfnstTrSetIdx is specified in Table 8-20 - Specification of lfnstTrSetIdx depending on predModeIntra.	
Table 8-20 - Specification of lfnstTrSetIdx	
predModeIntra	lfnstTrSetIdx
predModeIntra < 0	1
0 <= predModeIntra <= 1	0
2 <= predModeIntra <= 12	1
13 <= predModeIntra <= 23	2
24 <= predModeIntra <= 44	3
45 <= predModeIntra <= 55	2
56 <= predModeIntra <= 80	1

[0279] Table 12 shows a transform set for the LFNST and an LFNST transform set derived based on an input value for deriving a transform kernel matrix and an intra prediction mode.

[0280] As shown in Table 12, the transform kernel matrix (lowFreqTransMatrix) can be derived using a variable nTrS indicating the transform output size to derive the transform kernel matrix, intra prediction mode information (predModeIntra) for selection of the LFNST transform set, and the LFNST index signaled from the coding unit as input values.

[0281] There are four LFNST transform sets, such as 0, 1, 2, and 3, and the same transform set can be applied to modes located in mutually symmetric directions based on the symmetry of the intra prediction mode. When the intra prediction mode is non-directional planar mode or DC mode ($0 \leq \text{predModeIntra} \leq 1$), then the transform set is 0, in the case of wide-angle intra prediction mode ($\text{predModeIntra} < 0$, $56 \leq \text{predModeIntra} \leq 80$), the transform set is 1. On the other hand, as described above, when the CCLM is applied to the chroma block, the intra prediction mode (predModeIntra) of the chroma block for deriving the transform set can be set to the intra mode of the corresponding luma block (IntraPredMode Y [xTbY+nTbW/2] [yTbY+nTbH/2]) rather than 81 to 83 indicating the CCLM or the planar mode.

[0282] Accordingly, 81 to 83 are omitted in the intra prediction mode (predModeIntra) for transform set selection of Table 12.

[0283] Meanwhile, Table 13 below shows ctxInc allocated to the bin index of the tu_mts_idx syntax element described above (Assignment of ctxInc to syntax elements with context coded bins).

TABLE 13

Syntax element	binIdx					
	0	1	2	3	4	>=5
...	...	...	...	...	...	...
tu_cbf_cr[][]	tu_cbf_cb[][]	na	na	na	na	na
cu_qp_delta_abs	0	1	1	1	1	by-pass

TABLE 13-continued

Syntax element	binIdx					
	0	1	2	3	4	>=5
cu_qp_delta_sign_flag	bypass	na	na	na	na	na
transform_skip_flag[][]	0	na	na	na	na	na
tu_mts_idx[][]	0	1	2	3	na	na
tu_joint_cbr_residual[][]	0	na	na	na	na	na

[0284] As shown in Table 13, ctxInc of the first bin (binIdx=0) of tu_mts_idx is 0, ctxInc of the second bin (binIdx=1) is 1, and ctxInc of the third bin (binIdx=2) is 2 and the fourth ctxInc of a bin (binIdx=3) is 3.

[0285] In the conventional case, any one ctxInc is selected from among a plurality of ctxInc according to a predetermined condition and allocated to the first bin, as described above, by allocating one fixed ctxInc to the first bin without complying with a specific condition, coding efficiency can be increased.

[0286] The following drawings are provided to describe specific examples of the present disclosure. Since the specific designations of devices or the designations of specific signals/messages/fields illustrated in the drawings are provided for illustration, technical features of the present disclosure are not limited to specific designations used in the following drawings.

[0287] FIG. 16 is a flowchart illustrating an operation of a video decoding apparatus according to an embodiment of the present disclosure.

[0288] Each operation illustrated in FIG. 16 may be performed by the decoding apparatus 300 illustrated in FIG. 3. Specifically, S1610 to S1650 may be performed by the entropy decoder 310 illustrated in FIG. 3, S1620 may be performed by the dequantizer 321 illustrated in FIGS. 3, S1660 and S1670 may be performed by the inverse transformer 322 illustrated in FIGS. 3 and S1680 may be performed by the adder 340 illustrated in FIG. 3. Operations according to S1610 to S1680 are based on some of the foregoing details explained with reference to FIG. 4 to FIG. 15. Therefore, a description of specific details overlapping with those explained above with reference to FIG. 3 to FIG. 15 will be omitted or will be made briefly.

[0289] The decoding apparatus 300 according to an embodiment receives a bitstream including residual information, and may derive residual information about a current block, that is, a transform block to be transformed, eg, quantized transform coefficients, from the bitstream (S1610).

[0290] More specifically, the decoding apparatus 300 may decode information on quantized transform coefficients for the target block from the bitstream and may derive the quantized transform coefficients for the current block based on the information on the quantized transform coefficients for the current block. The information on the quantized transform coefficients for the target block may be included in a sequence parameter set (SPS) or a slice header and may include at least one of information on whether a reduced transform (RST) is applied, information on the simplification factor, information on a minimum transform size in which the reduced transform is applied, information on a maximum transform size in which the reduced transform is applied, a reduced inverse transform size, and information on a transform index indicating any one of transform kernel matrices included in a transform set.

[0291] The decoding apparatus 300 may perform dequantization on the quantized transform coefficients of the current block to derive transform coefficients (S1620).

[0292] The derived transform coefficients may be two-dimensionally arranged in the current block, and the decoding apparatus may derive the non-zero data, ie, information on non-zero significant coefficients in the current block, through the residual coding. That is, the decoding apparatus may determine the last position information of the non-zero significant coefficient in the current block.

[0293] The transform coefficients derived based on the residual information of S1620 may be the dequantized transform coefficients as described above, or may be quantized transform coefficients. That is, the transform coefficients may be data capable of checking whether the non-zero data in the current block regardless of quantization or not.

[0294] The decoding apparatus may derive a first variable indicating whether a significant coefficient exists in a region other than a DC region (a region in which a DC component is located) of at least one transform block among transform blocks constituting the coding block (S1630).

[0295] In this document, according to an example, the DC region may mean only a position with respect to a DC transform coefficient, that is, only a top-left position of a transform block.

[0296] The first variable may be a variable LfnstDcOnly that may be derived in a residual coding process. The first variable may be derived as 0 when the index of the subblock including the last significant coefficient in the current block is 0 and the position of the last significant coefficient in the subblock is greater than 0, and if the first variable is 0, the LFNST index can be parsed.

[0297] The first variable may be initially set to 1, and may be maintained at 1 or may be changed to 0 depending on whether the significant coefficient exists in the region other than the DC region.

[0298] According to an example, a variable log 2SbW and a variable log 2SbH indicating the height and width of the sub-block may be derived based on the size of the transform block, and numSbCoeff representing the number of coefficients that may exist in the subblock may be set based on the

variable log 2SbW and the variable log 2SbH [numSbCoeff=1«(log 2SbW+log 2SbH)].

[0299] The variable lastScanPos indicating the position of the last significant coefficient within the subblock is initially set to numSbCoeff, and the variable lastSubBlock indicating the subblock in which the last non-zero coefficient exists is initially “(1«(log 2TbWidth+log 2TbHeight-(log 2SbW+log 2SbH)))-set to 1”

[0300] While scanning in the diagonal direction in the subblock corresponding to lastSubBlock [lastScanPos--], it is checked whether the last significant coefficient other than 0 exists at the corresponding position.

[0301] when a significant coefficient is not found until the variable lastScanPos becomes 0 within the subblock pointed to by lastSubBlock, the variable lastScanPos is set to numSbCoeff again, and the variable lastSubBlock is also changed to the next sub-block in the scan direction [lastSubBlock--].

[0302] That is, as the variable lastScanPos and the variable lastSubBlock are updated according to the scan direction, the position where the last non-zero coefficient exists is identified.

[0303] The variable LfnstDcOnly indicates whether a non-zero coefficient exists at a position that is not a DC component for at least one transform block in one coding unit, when the non-zero coefficient exists in the position that is not the DC component for at least one transform block in the one coding unit, it becomes 0, and when non-zero coefficients do not exist at positions other than the DC components for all transform blocks in the one coding unit it becomes 1.

[0304] As shown in Table 8, when the index of the subblock in which the last non zero coefficient exists is 0 [lastSubBlock==0], and the position of the last nonzero coefficient in the subblock is greater than 0 [lastScanPos>0], the variable LfnstDcOnly may be derived as 0. The variable LfnstDcOnly can be derived as 0 only when the width and height of the transform block are 4 or more [log 2TbWidth>=2 && log 2TbHeight>=2], and no transform skip is applied [!transform_skip_flag[x0][y0]].

[0305] According to an example, the decoding apparatus may derive a second variable indicating whether a significant coefficient exists in the second region other than the first region at top-left of the current block (S1640).

[0306] The second variable may be a variable LfnstZeroOutSigCoeffFlag that may indicate that zero-out is performed when the LFNST is applied. The second variable is initially set to 1, and when a significant coefficient exists in the second region, the second variable may be changed to 0.

[0307] The variable LfnstZeroOutSigCoeffFlag can be derived as 0 when the index of the subblock in which the last non-zero coefficient exists is greater than 0 and the width and height of the transform block are both greater than 4 [(lastSubBlock>0 && log 2TbWidth>=2 && log 2TbHeight>=2)], or when the last position of the non-zero coefficient within the subblock in which the last non-zero coefficient exists is greater than 7 and the size of the transform block is 4x4 or 8x8 [(lastScanPos>7 && (log 2TbWidth==2|| log 2TbHeight==3)) && log 2TbWidth==log 2TbHeight)].

[0308] That is, in the transform block, when non-zero coefficients are derived from regions other than the top-left region where LFNST transform coefficients can exist, or for 4x4 blocks and 8x8 blocks when the non-zero coefficient

exists outside the 8th position in the scan order, the variable `LfnstZeroOutSigCoeffFlag` is set to 0.

[0309] The first region may be derived based on the size of the current block.

[0310] For example, when the size of the current block is 4×4 or 8×8 , the first region may be from the top-left of the current block to the eighth sample position in the scan direction.

[0311] When the size of the current block is 4×4 or 8×8 , since 8 data are output through the forward LFNST, the 8 transform coefficients received by the decoding apparatus can be arranged from the top-left of the current block to the 8th sample position in the scan direction as in FIG. 13(a) and FIG. 14(a).

[0312] Also, when the size of the current block is not 4×4 or 8×8 , the first region may be a 4×4 area at the top-left of the current block. If the size of the current block is not 4×4 or 8×8 , since 16 data are output through the forward LFNST, 16 transform coefficients received by the decoding apparatus may be arranged in the top-left 4×4 area of the current block as shown in FIGS. 13(b) to (d) and FIG. 14(b).

[0313] Meanwhile, transform coefficients that may be arranged in the first region may be arranged along a diagonal scan direction as shown in FIG. 8.

[0314] Also, according to an example, the maximum number of transform coefficients that may exist in a block to which the LFNST is applied may be 16.

[0315] When the first variable indicates that the significant coefficient exists in the region other than the DC region, and the second variable indicates that there is no significant coefficient in the second region, the decoding apparatus can parse the LFNST index from the bitstream (S1650).

[0316] That is, when the first variable that was set to 1 is changed to 0 and the second variable is maintained at 1, the LFNST index may be parsed. In other words, there is the significant coefficient in the sub-block including the DC region, that is, the top-left 4×4 block, in addition to the DC region, and when the significant coefficient does not exist by checking the significant coefficient up to the second region of the current block, the LFNST index for the LFNST may be parsed.

[0317] In summary, at the coding unit level, the first variable `LfnstDcOnly` and the second variable `LfnstZeroOutSigCoeffFlag` are set to 1, respectively, and then are newly derived through the process shown in Table 8 at the residual coding level. Only when the variable `LfnstDcOnly` derived from the residual coding level is 0 and the variable `LfnstZeroOutSigCoeffFlag` is 1 [if (`LfnstDcOnly`=0 & & `LfnstZeroOutSigCoeffFlag`=1)], `lfnst_idx` may be signaled at the coding unit level.

[0318] As described above, when the forward LFNST is performed by the encoding apparatus, the zero-out in which the remaining regions of the current block except for the region in which the LFNST transform coefficients may exist may be performed as 0.

[0319] Therefore, when the significant coefficient exists in the second region, it is certain that the LFNST is not applied, so the LFNST index is not signaled and the decoding apparatus does not parse the LFNST index.

[0320] The LFNST index information is received as syntax information and the syntax information is received as a binarized bin string including 0 and 1.

[0321] The syntax element of the LFNST index according to this embodiment may indicate whether an inverse LFNST

or an inverse non-separable transform is applied and any one of a transform kernel matrix included in the transform set, and the transform set includes two transform kernel matrices. In this case, the syntax element of the transform index may have three values.

[0322] That is, according to an embodiment, the syntax element value for the LFNST index may include 0 indicating a case in which the inverse LFNST is not applied to the target block, 1 indicating the first transformation kernel matrix among the transformation kernel matrices, and 2 indicating the second transform kernel matrix among the transformation kernel matrices.

[0323] When the LFNST index is parsed, the decoding apparatus may apply the LFNST matrix to the transform coefficients of the first region to derive modified transform coefficients (S1660).

[0324] The inverse transformer 332 of the decoding apparatus 300 may determine a transform set based on a mapping relationship according to an intra prediction mode applied to a current block, and may perform an inverse LFNST, that is, the inverse non-separable transformation based on the transform set and the values of syntax elements for the LFNST index.

[0325] As described above, a plurality of transform sets may be determined according to an intra prediction mode of a transform block to be transformed, and an inverse LFNST may be performed based on any one of transform kernel matrices, that is, LFNST matrices included in a transform set indicated by an LFNST index. A matrix applied to the inverse LFNST may be named as an inverse LFNST matrix or an LFNST matrix, and the name of this matrix is irrelevant as long as it has a transpose relation with the matrix used for the forward LFNST.

[0326] In one example, the inverse LFNST matrix may be a non-square matrix in which the number of columns is less than the number of rows.

[0327] Meanwhile, a predetermined number of modified transform coefficients may be derived based on the size of the current block. For example, when the height and width of the current block are 8 or more, 48 modified transform coefficients are derived as shown on the left of FIG. 7. And when the width and height of the current block are not equal to or greater than 8, that is, when the width and height of the current block are greater than or equal to 4 and the width or height of the current block is less than 8, 16 modified transform coefficients may be derived as shown on the right of FIG. 7.

[0328] As shown in FIG. 7, the 48 modified transform coefficients may be arranged in the top-left, top-right, and bottom-left 4×4 regions of the top-left 8×8 region of the current block, and the 16 modified transform coefficients may be arranged in the top-left 4×4 region of the current block.

[0329] The 48 modified transform coefficients and the 16 modified transform coefficients may be arranged in a vertical or horizontal direction according to the intra prediction mode of the current block. For example, when the intra prediction mode is a horizontal direction (modes 2 to 34 in FIG. 9) based on a diagonal direction (mode 34 in FIG. 9), the modified transform coefficients may be arranged in the horizontal direction, that is, in the row-first direction, as shown in (a) of FIG. 7. When the intra prediction mode is a vertical direction (modes 35 to 66 in FIG. 9) based on a diagonal direction, the modified transform coefficients may

be arranged in the vertical direction, that is, in a column-first direction, as shown in (b) of FIG. 7.

[0330] In one embodiment, S1660 may include decoding the transform index, determining whether it corresponds to the conditions to apply the inverse RST based on the transform index, that is, the LFNST index, selecting a transform kernel matrix and applying the inverse LFNST to the transform coefficients based on the selected transform kernel matrix and/or the simplification factor when the conditions for applying the inverse LFNST is satisfied. In this case, the size of the simplification inverse transform matrix may be determined based on the simplification factor.

[0331] Referring to S1660, it can be confirmed that residual samples for the target block are derived based on the inverse LFNST of the transform coefficients for the target block. Regarding the size of an inverse transform matrix, the size of a general inverse transform matrix is $N \times N$, while the size of an inverse LFNST matrix is reduced to $N \times R$, thus making it possible to reduce memory occupancy by an R/N ratio when performing the inverse LFNST compared to when performing a general transform. Further, compared to the number of multiplication operations, $N \times N$, when using the general inverse transform matrix, it is possible to reduce the number of multiplication operations by an R/N ratio (to $N \times R$) when the inverse LFNST matrix is used. In addition, since only R transform coefficients need to be decoded when the inverse LFNST is applied, the total number of transform coefficients for the target block may be reduced from N to R , compared to when the general inverse transform is applied in which N transform coefficients need to be decoded, thus increasing decoding efficiency. That is, according to S1660, (inverse) transform efficiency and decoding efficiency of the decoding apparatus 300 may be increased through the inverse LFNST.

[0332] The decoding apparatus 300 according to an embodiment may derive residual samples for the target block based on the inverse primary transform of the modified transform coefficients (S1670).

[0333] On the other hand, when the LFNST is not applied, only the MTS-based primary inverse transform procedure may be applied in the inverse transform procedure as follows. That is, the decoding apparatus may determine whether the LFNST is applied to the current block as in the above-described embodiment, and when the LFNST is not applied, the decoding apparatus may derive residual samples from transform coefficients through a primary inverse transform.

[0334] The decoding apparatus may determine that the LFNST is not applied when significant coefficients exist in the second region other than the first top-left region of the current block, and may derive residual samples from the transform coefficients through primary inverse transform.

[0335] The primary inverse transform procedure may be referred to as an inverse primary transform procedure or an inverse MTS transform procedure. Such an MTS-based primary inverse transform procedure may also be omitted in some cases.

[0336] In addition, a simplified inverse transform may be applied to the inverse primary transform, or a conventional separable transform may be used.

[0337] The decoding apparatus 300 according to an embodiment may generate reconstructed samples based on residual samples of the current block and prediction samples of the current block (S1680).

[0338] The following drawings are provided to describe specific examples of the present disclosure. Since the specific designations of devices or the designations of specific signals/messages/fields illustrated in the drawings are provided for illustration, technical features of the present disclosure are not limited to specific designations used in the following drawings.

[0339] FIG. 17 is a flowchart illustrating an operation of a video encoding apparatus according to an embodiment of the present disclosure.

[0340] Each operation illustrated in FIG. 17 may be performed by the encoding apparatus 200 illustrated in FIG. 2. Specifically, S1710 may be performed by the predictor 220 illustrated in FIG. 2, S1720 may be performed by the subtractor 231 illustrated in FIGS. 2, S1730 to S1750 may be performed by the transformer 232 illustrated in FIGS. 2, and S1760 and S1770 may be performed by the quantizer 233 and the entropy encoder 240 illustrated in FIG. 2. Operations according to S1710 to S1770 are based on some of contents described in FIG. 4 to FIG. 15. Therefore, a description of specific details overlapping with those explained above with reference to FIG. 2 and FIG. 4 to FIG. 15 will be omitted or will be made briefly.

[0341] The encoding apparatus 200 according to an embodiment may derive prediction samples based on the intra prediction mode applied to the current block (S1710).

[0342] The encoding apparatus 200 according to an embodiment may derive residual samples for the current block based on the prediction samples (S1720).

[0343] The encoding apparatus 200 according to an embodiment may derive transform coefficients for the target block based on a primary transform for the residual samples (S1730).

[0344] The primary transform may be performed through a plurality of transform kernels, and in this case, a transform kernel may be selected based on the intra prediction mode.

[0345] The encoding apparatus 200 may determine whether to perform a secondary transform or a non-separable transform, specifically, LFNST, on transform coefficients for the current block.

[0346] When it is determined to perform the LFNST, the encoding apparatus 200 may derive modified transform coefficients for the current block based on the transform coefficients of the first region at the top-left of the current block and a predetermined LFNST matrix (S1740).

[0347] The encoding apparatus 200 may determine a transform set based on a mapping relationship according to an intra prediction mode applied to a current block, and may perform the LFNST, that is, a non-separable transform based on one of two LFNST matrices included in the transform set.

[0348] As described above, a plurality of transform sets may be determined according to the intra prediction mode of a transform block to be transformed. A matrix applied to the LFNST has a transpose relationship with a matrix used for the inverse LFNST.

[0349] In one example, the LFNST matrix may be a non-square matrix in which the number of rows is less than the number of columns.

[0350] The first region may be derived based on the size of the current block. For example, when the height and width of the current block is greater than or equal to 8, the first region is a 4×4 area of the top-left, top-right, and bottom-left of the 8×8 area at the top-left of the current block as shown in the left of FIG. 7. When the height and width of the

current block are not equal to or greater than 8, the first region may be a 4×4 area at the top-left of the current block as shown in the right of FIG. 7.

[0351] The transform coefficients of the first region may be one-dimensionally arranged in a vertical or horizontal direction according to the intra prediction mode of the current block for a multiplication operation with the LFNST matrix.

[0352] The 48 modified transform coefficients or the 16 modified transform coefficients of the first region may be read in the vertical or horizontal direction according to the intra prediction mode of the current block and arranged in one dimension. For example, if the intra prediction mode is a horizontal direction (modes 2 to 34 in FIG. 9) based on a diagonal direction (mode 34 in FIG. 9), the transform coefficients may be arranged in the horizontal direction, that is, in the row-first direction, as shown in (a) of FIG. 7. When the intra prediction mode is a vertical direction (modes 35 to 66 in FIG. 9) based on the diagonal direction, the transform coefficients may be arranged in the vertical direction, that is, in a column-first direction, as shown in (b) of FIG. 7.

[0353] In one example, the LFNST may be performed based on a simplified transform matrix or a transform kernel matrix, and the simplified transform matrix may be a non-square matrix in which the number of rows is less than the number of columns.

[0354] In one embodiment, S1740 may include determining whether the conditions for applying the LFNST are satisfied, generating and encoding the LFNST index based on the determination, selecting a transform kernel matrix and applying the LFNST to residual samples based on the selected transform kernel matrix and/or the simplification factor when the conditions for applying LFNST is satisfied. In this case, the size of the simplification transform matrix may be determined based on the simplification factor.

[0355] Referring to S1740, it can be confirmed that transform coefficients for the target block are derived based on the LFNST for the residual samples. Regarding the size of a transform kernel matrix, the size of a general transform kernel matrix is N×N, while the size of a simplified transform matrix is reduced to R x N, thus making it possible to reduce memory occupancy by an R/N ratio when performing the RST compared to when performing a general transform. Further, compared to the number of multiplication operations, N×N, when using the general transform kernel matrix, it is possible to reduce the number of multiplication operations by an R/N ratio (to R x N) when the simplified transform kernel matrix is used. In addition, since only R transform coefficients are derived when the RST is applied, the total number of transform coefficients for the target block may be reduced from N to R, compared to when the general transform is applied in which N transform coefficients are derived, thus reducing the amount of data transmitted by the encoding apparatus 200 to the decoding apparatus 300. That is, according to S1740, transform efficiency and coding efficiency of the encoding apparatus 200 may be increased through the LFNST.

[0356] Meanwhile, according to an example, the encoding apparatus may zero out the second region of the current block in which the modified transform coefficients do not exist (S1750).

[0357] As FIGS. 13 and 14, all remaining regions of the current block in which the modified transform coefficients do not exist may be treated as zeros. Due to the zero-out, the

amount of computation required to perform the entire transform process is reduced, and the amount of computation required for the entire transform process is reduced, thereby reducing power consumption required to perform the transform. In addition, the image coding efficiency may be increased by reducing the latency involved in the transform process.

[0358] On the other hand, when the LFNST is not applied, only the MTS-based primary transform procedure may be applied in the transform procedure as described above. That is, the encoding apparatus may determine whether the LFNST is applied to the current block as in the above-described embodiment, and when the LFNST is not applied, the encoding apparatus may derive transform coefficients from residual samples through the primary transform.

[0359] This primary transform procedure may be referred to as a primary transform procedure or an MTS transform procedure. Such an MTS-based primary transform procedure may also be omitted in some cases.

[0360] The encoding apparatus according to an example may configure image information such that an LFNST index indicating an LFNST matrix is parsed when a significant coefficient exists in a region excluding a DC region of the current block and the above-described zero-out is performed (S1760).

[0361] The encoding apparatus may configure the image information so that the image information shown in Tables 6 and 8 may be parsed by the decoding apparatus.

[0362] According to an example, when the index of the subblock including the last significant coefficient in the current block is 0 and the position of the last significant coefficient in the subblock is greater than 0, it is determined that the significant coefficient exists in the region other than the DC region, and the image information can be configured such that the LFNST index is signaled. In this document, the first position in the scan order may be 0.

[0363] In addition, according to an example, when the index of the sub-block including the last significant coefficient in the current block is greater than 0 and the width and height of the current block are 4 or more, it is determined that it is certain that the LFNST is not applied, and the image information may be configured so that the LFNST index is not signaled.

[0364] In addition, according to an example, when the size of the current block is 4×4 or 8×8 and the start of the position in the scan order is 0, the position of the last significant coefficient is greater than 7, the encoding apparatus may determine that it is certain that the LFNST is not applied, and configure the image information so that the LFNST index is not signaled.

[0365] That is, the encoding apparatus can configure the image information so that the LFNST index can be parsed according to the derived variable value after the variable LfnstDcOnly and the variable LfnstZeroOutSigCoeffFlag are derived in the decoding apparatus.

[0366] The encoding apparatus 200 according to an embodiment may derive quantized transform coefficients by performing quantization based on the modified transform coefficients for the target block, and may encode and output image information including information about the quantized transform coefficients and the LFNST index (S1770).

[0367] That is, the encoding apparatus may generate residual information including information on quantized transform coefficients. The residual information may include

the above-described transform related information/syntax element. The encoding apparatus may encode image/video information including residual information and output the encoded image/video information in the form of a bitstream.

[0368] More specifically, the encoding apparatus 200 may generate information about the quantized transform coefficients and encode the information about the generated quantized transform coefficients.

[0369] In one example, information on the quantized transform coefficients may include at least one of information on whether LFNST is applied, information on a simplification factor, information on a minimum transform size to which LFNST is applied, and information on a maximum transform size to which LFNST is applied.

[0370] Also, the encoding apparatus 200 may encode information on the size of the maximum transform applied block, for example, syntax information on the transform block size such as `sps_log2_max_luma_transform_size_minus5`, or flag information such as `sps_max_luma_transform_size_64_flag` at the sequence parameter set level.

[0371] In the present disclosure, at least one of quantization/dequantization and/or transform/inverse transform may be omitted. When quantization/dequantization is omitted, a quantized transform coefficient may be referred to as a transform coefficient. When transform/inverse transform is omitted, the transform coefficient may be referred to as a coefficient or a residual coefficient, or may still be referred to as a transform coefficient for consistency of expression.

[0372] In addition, in the present disclosure, a quantized transform coefficient and a transform coefficient may be referred to as a transform coefficient and a scaled transform coefficient, respectively. In this case, residual information may include information on a transform coefficient(s), and the information on the transform coefficient(s) may be signaled through a residual coding syntax. Transform coefficients may be derived based on the residual information (or information on the transform coefficient(s)), and scaled transform coefficients may be derived through inverse transform (scaling) of the transform coefficients. Residual samples may be derived based on the inverse transform (transform) of the scaled transform coefficients. These details may also be applied/expressed in other parts of the present disclosure.

[0373] In the above-described embodiments, the methods are explained on the basis of flowcharts by means of a series of steps or blocks, but the present disclosure is not limited to the order of steps, and a certain step may be performed in order or step different from that described above, or concurrently with another step. Further, it may be understood by a person having ordinary skill in the art that the steps shown in a flowchart are not exclusive, and that another step may be incorporated or one or more steps of the flowchart may be removed without affecting the scope of the present disclosure.

[0374] The above-described methods according to the present disclosure may be implemented as a software form, and an encoding apparatus and/or decoding apparatus according to the disclosure may be included in a device for image processing, such as, a TV, a computer, a smartphone, a set-top box, a display device or the like.

[0375] When embodiments in the present disclosure are embodied by software, the above-described methods may be embodied as modules (processes, functions or the like) to perform the above-described functions. The modules may be

stored in a memory and may be executed by a processor. The memory may be inside or outside the processor and may be connected to the processor in various well-known manners. The processor may include an application-specific integrated circuit (ASIC), other chipset, logic circuit, and/or a data processing device. The memory may include a read-only memory (ROM), a random access memory (RAM), a flash memory, a memory card, a storage medium, and/or other storage device. That is, embodiments described in the present disclosure may be embodied and performed on a processor, a microprocessor, a controller or a chip. For example, function units shown in each drawing may be embodied and performed on a computer, a processor, a microprocessor, a controller or a chip.

[0376] Further, the decoding apparatus and the encoding apparatus to which the present disclosure is applied, may be included in a multimedia broadcasting transceiver, a mobile communication terminal, a home cinema video device, a digital cinema video device, a surveillance camera, a video chat device, a real time communication device such as video communication, a mobile streaming device, a storage medium, a camcorder, a video on demand (VoD) service providing device, an over the top (OTT) video device, an Internet streaming service providing device, a three-dimensional (3D) video device, a video telephony video device, and a medical video device, and may be used to process a video signal or a data signal. For example, the over the top (OTT) video device may include a game console, a Blu-ray player, an Internet access TV, a Home theater system, a smartphone, a Tablet PC, a digital video recorder (DVR) and the like.

[0377] In addition, the processing method to which the present disclosure is applied, may be produced in the form of a program executed by a computer, and be stored in a computer-readable recording medium. Multimedia data having a data structure according to the present disclosure may also be stored in a computer-readable recording medium. The computer-readable recording medium includes all kinds of storage devices and distributed storage devices in which computer-readable data are stored. The computer-readable recording medium may include, for example, a Blu-ray Disc (BD), a universal serial bus (USB), a ROM, a PROM, an EPROM, an EEPROM, a RAM, a CD-ROM, a magnetic tape, a floppy disk, and an optical data storage device. Further, the computer-readable recording medium includes media embodied in the form of a carrier wave (for example, transmission over the Internet). In addition, a bitstream generated by the encoding method may be stored in a computer-readable recording medium or transmitted through a wired or wireless communication network. Additionally, the embodiments of the present disclosure may be embodied as a computer program product by program codes, and the program codes may be executed on a computer by the embodiments of the present disclosure. The program codes may be stored on a computer-readable carrier.

[0378] FIG. 18 illustrates the structure of a content streaming system to which the present disclosure is applied.

[0379] Further, the contents streaming system to which the present disclosure is applied may largely include an encoding server, a streaming server, a web server, a media storage, a user equipment, and a multimedia input device.

[0380] The encoding server functions to compress to digital data the contents input from the multimedia input devices, such as the smart phone, the camera, the camcorder

and the like, to generate a bitstream, and to transmit it to the streaming server. As another example, in a case where the multimedia input device, such as, the smart phone, the camera, the camcorder or the like, directly generates a bitstream, the encoding server may be omitted. The bitstream may be generated by an encoding method or a bitstream generation method to which the present disclosure is applied. And the streaming server may store the bitstream temporarily during a process to transmit or receive the bitstream.

[0381] The streaming server transmits multimedia data to the user equipment on the basis of a user's request through the web server, which functions as an instrument that informs a user of what service there is. When the user requests a service which the user wants, the web server transfers the request to the streaming server, and the streaming server transmits multimedia data to the user. In this regard, the contents streaming system may include a separate control server, and in this case, the control server functions to control commands/responses between respective equipments in the content streaming system.

[0382] The streaming server may receive contents from the media storage and/or the encoding server. For example, in a case the contents are received from the encoding server, the contents may be received in real time. In this case, the streaming server may store the bitstream for a predetermined period of time to provide the streaming service smoothly.

[0383] For example, the user equipment may include a mobile phone, a smart phone, a laptop computer, a digital broadcasting terminal, a personal digital assistant (PDA), a portable multimedia player (PMP), a navigation, a slate PC, a tablet PC, an ultrabook, a wearable device (e.g., a watch-type terminal (smart watch), a glass-type terminal (smart glass), a head mounted display (HMD)), a digital TV, a desktop computer, a digital signage or the like. Each of servers in the contents streaming system may be operated as a distributed server, and in this case, data received by each server may be processed in distributed manner.

[0384] Claims disclosed herein can be combined in a various way. For example, technical features of method claims of the present disclosure can be combined to be implemented or performed in an apparatus, and technical features of apparatus claims can be combined to be implemented or performed in a method. Further, technical features of method claims and apparatus claims can be combined to be implemented or performed in an apparatus, and technical features of method claims and apparatus claims can be combined to be implemented or performed in a method.

What is claimed is:

1. An image decoding apparatus, comprising:

a memory; and

at least one processor connected to the memory, wherein the at least one processor is configured to:

obtain residual information from a bitstream;

derive transform coefficients for a current block based on the residual information;

derive a first variable related to whether a significant coefficient exists in a region excluding a Direct-Current (DC) region of the current block;

derive a second variable related to whether the significant coefficient exists in a second region other than a first region at a top-left of the current block;

parse a Low Frequency Non-Separable Transform (LFNST) index from the bitstream based on the first

variable being related to that the significant coefficient exists in the region excluding the DC region and the second variable being related to that the significant coefficient does not exist in the second region;

derive modified transform coefficients by applying an LFNST matrix derived based on the LFNST index to the transform coefficients in the first region;

derive residual samples for the current block based on an inverse primary transform for the modified transform coefficients; and

generating a reconstructed picture based on the residual samples for the current block.

2. The image decoding apparatus of claim 1, wherein the first region is derived based on a size of the current block, wherein based on the size of the current block being 4×4 or 8×8, the first region is from the top-left of the current block to an 8th sample position in a scan direction, and wherein based on the size of the current block not being 4×4 or 8×8, the first region is a 4×4 area at the top-left of the current block.

3. The image decoding apparatus of claim 1, wherein a predetermined number of the modified transform coefficients are derived based on a size of the current block,

wherein based on a height and a width of the current block being greater than or equal to 8, 48 modified transform coefficients are derived, and

wherein based on the width and the height of the current block being 4 or more and the width or the height of the current block is less than 8, 16 modified transform coefficients are derived.

4. An image encoding apparatus, comprising:

a memory; and

at least one processor connected to the memory, wherein the at least one processor is configured to:

derive prediction samples for a current block;

derive residual samples for the current block based on the prediction samples;

derive transform coefficients for the current block based on a primary transform for the residual samples;

derive modified transform coefficients for the current block based on the transform coefficients of a first region at a top-left of the current block and a predetermined Low Frequency Non-Separable Transform (LFNST) matrix;

zero-out a second region of the current block in which the modified transform coefficients do not exist;

construct image information so that an LFNST index related to the LFNST matrix is signaled based on a significant coefficient existing in a region excluding a Direct-Current (DC) region of the current block and the zeroing-out being performed, and

encode the image information including residual information derived through quantization of the modified transform coefficients and the LFNST index.

5. The image encoding apparatus of claim 4, wherein the first region is derived based on a size of the current block, wherein based on a height and a width of the current block being greater than or equal to 8, the first region is a top-left, a top-right, and a bottom-left 4×4 areas within a top-left 8×8 area of the current block, and

wherein based on the width and the height of the current block being 4 or more and the width or the height of the current block being less than 8, the first region is the top-left 4×4 area of the current block.

- 6. The image encoding apparatus of claim 4, wherein a predetermined number of the modified transform coefficients are derived based on a size of the current block,
 - wherein based on the size of the current block being 4×4 or 8×8, 8 modified transform coefficients are derived, and
 - wherein based on the size of the current block not being 4×4 or 8×8, 16 modified transform coefficients are derived.
- 7. An apparatus for transmitting data for an image, the method comprising:
 - at least one processor configured to generate a bitstream for the image information, wherein the bitstream is generated based on deriving prediction samples for a current block; deriving residual samples for the current block based on the prediction samples; deriving transform coefficients for the current block based on a primary transform for the residual samples; deriving

modified transform coefficients for the current block based on the transform coefficients of a first region at a top-left of the current block and a predetermined Low Frequency Non-Separable Transform (LFNST) matrix; zeroing-out a second region of the current block in which the modified transform coefficients do not exist; constructing image information so that an LFNST index related to the LFNST matrix is signaled based on a significant coefficient existing in a region excluding a Direct-Current (DC) region of the current block and the zeroing-out being performed, and encoding the image information including residual information derived through quantization of the modified transform coefficients and the LFNST index to generate the bitstream; and

a transmitter configured to transmit the data comprising the bitstream.

* * * * *